

MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Tarea 1 - Módulo 10



Campus Internacional
CIBERSEGURIDAD



UCAM
UNIVERSIDAD
CATÓLICA DE MURCIA

- 1. Introducción
 - 1.1 Qué es Ring 0
 - 1.2 Diferencia entre modo usuario y modo kernel
 - 1.3 Por qué un driver malicioso es peligroso
 - 1.4 Por qué WinDbg es útil para inspeccionar el kernel
 - 1.5 Objetivo de este trabajo
- 2. Fundamento teórico
 - 2.1 Qué es un driver de kernel
 - 2.2 Qué es un **DRIVER_OBJECT**
 - 2.3 Qué estructuras del kernel vamos a observar
 - 2.4 Relación con técnicas de rootkit
- 3. Entorno de laboratorio
 - 3.1 Sistema operativo Windows usado
 - 3.2 Máquina host física: Kali Linux con VMware
 - 3.3 Máquina analista Windows con WinDbg
 - 3.3.1 Carga de los símbolos
 - 3.3.2 Localización de kdnet.exe
 - 3.4 Máquina destino Windows 11 actualizada
 - 3.5 Configuración de red para Remote Kernel Debugging
 - 3.6 Configuración de una conexión de depuración remota
- 4. Detección manual con WinDbg
 - 4.1. Configuración de símbolos
 - 4.2 Mostramos información básica del sistema que estamos depurando
 - 4.3 Mostramos información detallada del módulo nt
 - 4.4 Analizamos los drivers cargados
 - 4.5 Comprobamos el acceso a estructuras internas del kernel
 - 4.6 Probamos algunas estructuras para ver si todo funciona bien
 - 4.7 Listamos el directorio de objetos de drivers
 - 4.8 Probamos con el driver WdNisDrv
 - 4.9. Listamos los procesos activos desde la perspectiva del kernel.
 - 4.6 Indicadores de anomalía
- 5. Caso de estudio: **mimidrv.sys**
 - 5.1 Qué es Mimikatz
 - 5.2 Qué es mimidrv.sys
 - 5.3 Qué relación tiene con LSASS/PPL
 - 5.4 Ejecución controlada de Mimikatz/mimidrv.sys y análisis desde WinDbg
 - A) Tomamos una línea base antes de ejecutar Mimikatz
 - B) Deshabilitamos antivirus en maquina destino
 - C) Copiamos Mimikatz a la máquina destino
 - D) Ejecutamos Minikatz
 - E) Ejecutamos WinDbg en la maquina analista**
 - F) Confirmar objeto de driver en WinDbg
 - G) Ver las Dispatch routines
 - H) Confirmar rango de memoria del driver
 - I) Confirmar rango de memoria del driver
 - 5.7 Tabla de resultados manuales en WinDbg

- **6. Automatización mediante scripting**
 - **6.1. Objetivo del script**
 - **6.2 El script**
 - **6.3. Funcionamiento del script**
 - **6.4. Conclusiones del funcionamiento del script**
- **7. Conclusiones**
 - **7.1 Qué se puede detectar**
 - **7.2 Limitaciones**

1. Introducción

El análisis de malware en sistemas Windows no se limita únicamente al estudio de procesos, ficheros o comunicaciones visibles desde el espacio de usuario. Una parte especialmente relevante de las amenazas avanzadas se encuentra en aquellas **muestras capaces de interactuar con el sistema operativo a bajo nivel, modificando estructuras internas del kernel, cargando drivers maliciosos o alterando mecanismos de protección del propio sistema**. Este tipo de actividad resulta especialmente crítica porque se produce en una **zona privilegiada del sistema**, donde las **herramientas tradicionales de monitorización pueden no tener visibilidad suficiente**.

En este contexto, el presente trabajo tiene como objetivo estudiar el **uso de WinDbg como herramienta de análisis y depuración del kernel de Windows** para identificar posibles anomalías asociadas a código ejecutado en **Ring 0**. A través de la inspección de drivers, módulos cargados y estructuras internas del sistema operativo, se busca comprender cómo un analista puede detectar indicios de actividad sospechosa en modo kernel.

El enfoque del trabajo se centra en el análisis de drivers y estructuras del kernel, tomando como referencia el caso de **mimikatz.sys**, el **driver de Mimikatz**, como ejemplo de componente con capacidades sensibles en **Ring 0**. No se pretende realizar una infección real ni ejecutar malware en un entorno productivo, sino utilizar este caso como referencia técnica para comprender qué tipo de comportamientos pueden resultar relevantes desde el punto de vista defensivo.

1.1 Qué es Ring 0

Los procesadores modernos implementan distintos niveles de privilegio para separar las operaciones normales de las operaciones críticas del sistema. Estos niveles se conocen habitualmente como anillos de protección o *protection rings*. En arquitecturas **x86/x64**, el nivel más privilegiado es **Ring 0**, mientras que el nivel menos privilegiado utilizado habitualmente por las aplicaciones es **Ring 3**.

Ring 0 corresponde al modo kernel. En este nivel se ejecutan los componentes más críticos del sistema operativo, como el kernel de Windows, la capa de abstracción de hardware, los controladores o drivers y otros elementos encargados de gestionar recursos esenciales del sistema. Desde **Ring 0** es posible acceder directamente a memoria del kernel, interactuar con dispositivos hardware, gestionar interrupciones, modificar estructuras internas del sistema operativo y ejecutar instrucciones privilegiadas que no están disponibles para los procesos normales.

Por el contrario, las aplicaciones comunes se ejecutan en **Ring 3**, también conocido como **modo usuario**. En este modo, los procesos tienen un acceso limitado a los recursos del sistema y necesitan solicitar servicios al kernel mediante llamadas al sistema. Esta separación entre **modo usuario** y **modo kernel** es una medida de seguridad fundamental, ya que **evita que una aplicación normal pueda modificar directamente memoria crítica, interferir con otros procesos o alterar el funcionamiento global del sistema operativo**.

La ejecución de código en **Ring 0** implica, por tanto, un nivel de privilegio muy elevado. Por esta razón, los drivers legítimos deben estar correctamente diseñados y firmados, ya que un error en modo kernel puede provocar inestabilidad, corrupción de memoria o incluso una caída completa del sistema mediante una pantalla azul o **BSOD**. Del mismo modo, un driver malicioso o vulnerable puede ser utilizado para comprometer gravemente la seguridad del equipo.

Desde el punto de vista del malware, **Ring 0** resulta especialmente atractivo porque permite realizar acciones que serían mucho más difíciles o imposibles desde modo usuario. Un malware ejecutado en **modo kernel** podría ocultar procesos, manipular listas internas del sistema, interceptar llamadas al sistema, modificar estructuras como **EPROCESS** o **DRIVER_OBJECT**, alterar rutinas de drivers, desactivar mecanismos de protección o dificultar la detección por parte de herramientas de seguridad.

Este tipo de técnicas está relacionado con el funcionamiento de los rootkits, cuyo objetivo suele ser mantener persistencia, ocultar actividad maliciosa o modificar el comportamiento normal del sistema operativo. Por ejemplo, un rootkit podría alterar estructuras enlazadas del kernel para ocultar un proceso, modificar entradas de tablas internas para interceptar llamadas o cargar un driver que actúe como intermediario entre el malware y el sistema operativo.

Herramientas como **WinDbg** permiten **inspeccionar el estado interno del sistema**, consultar estructuras del kernel, revisar drivers cargados y analizar direcciones de memoria, proporcionando una visión mucho más profunda que la ofrecida por herramientas ejecutadas únicamente en modo usuario.

1.2 Diferencia entre modo usuario y modo kernel

Windows, al igual que otros sistemas operativos modernos, separa la ejecución del software en distintos niveles de privilegio con el objetivo de proteger la estabilidad y la seguridad del sistema. Esta separación permite que las aplicaciones comunes no puedan acceder directamente a recursos críticos, como la memoria del kernel, los dispositivos hardware o las estructuras internas del sistema operativo. En términos generales, esta división se representa mediante dos modos principales: el modo usuario y el modo kernel.

El modo usuario es el entorno en el que se ejecutan la mayoría de aplicaciones convencionales, como navegadores, editores de texto, herramientas de análisis, clientes de correo o procesos lanzados por el propio usuario. Estos procesos tienen privilegios limitados y no pueden acceder directamente al hardware ni modificar estructuras internas del sistema operativo. Cuando una aplicación en modo usuario necesita realizar una operación privilegiada, como leer un archivo, crear un proceso, abrir una conexión de red o reservar memoria, **debe solicitarlo al sistema operativo mediante llamadas al sistema**.

Esta separación actúa como una **barrera de seguridad**. Si una aplicación en modo usuario falla, normalmente el error afecta solo a ese proceso concreto. Por ejemplo, si un programa se bloquea o intenta acceder a una dirección de memoria no válida, el sistema operativo puede finalizar ese proceso sin comprometer el funcionamiento global del equipo. Esto permite aislar fallos y evitar que una aplicación defectuosa afecte directamente al kernel o al resto de procesos.

El modo kernel, por el contrario, es el entorno de ejecución más privilegiado del sistema operativo. En este modo se ejecutan componentes críticos como el kernel de Windows, la capa de abstracción de hardware, el gestor de memoria, el planificador de hilos y los drivers. Estos componentes necesitan un nivel de privilegio elevado porque son responsables de gestionar recursos fundamentales del sistema, como la memoria física, la CPU, los dispositivos, las interrupciones y las operaciones de entrada/salida.

La principal diferencia entre ambos modos se encuentra, por tanto, en el nivel de privilegio y en el acceso a los recursos. Mientras que **un proceso en modo usuario trabaja en un espacio de memoria virtual privado y aislado, el código que se ejecuta en modo kernel puede acceder a zonas de memoria compartidas y a estructuras internas del sistema operativo.** Esta capacidad es necesaria para que Windows funcione correctamente, pero también implica un riesgo elevado si el código que se ejecuta en kernel contiene errores o se comporta de forma maliciosa.

Otra diferencia importante está relacionada con la gestión de errores. **En modo usuario, un fallo suele provocar únicamente el cierre de la aplicación afectada. En modo kernel, en cambio, un error puede comprometer la estabilidad de todo el sistema.** Un acceso incorrecto a memoria, una mala gestión de punteros, una escritura en una estructura crítica o una operación inválida dentro de un driver pueden provocar una caída completa del sistema, normalmente reflejada en una pantalla azul o BSOD.

Desde el punto de vista de la seguridad, esta diferencia resulta fundamental. **Un malware ejecutado en modo usuario tiene limitaciones importantes: debe interactuar con el sistema a través de APIs,** puede ser monitorizado por herramientas de seguridad y no puede modificar directamente el núcleo del sistema operativo. Sin embargo, **un malware que consigue ejecutar código en modo kernel adquiere una posición mucho más privilegiada.** Desde ahí puede manipular estructuras internas, ocultar procesos, interceptar llamadas al sistema, modificar rutinas de drivers, alterar mecanismos de protección o dificultar su propia detección.

Los drivers son un ejemplo claro de código que se ejecuta en modo kernel. Aunque muchos drivers son legítimos y necesarios para el funcionamiento del sistema, también pueden ser abusados por atacantes. Un driver malicioso o vulnerable puede actuar como puerta de entrada al kernel, permitiendo ejecutar operaciones que no serían posibles desde modo usuario. Por este motivo, el análisis de drivers cargados, sus estructuras asociadas y sus rutinas internas es una parte relevante en la detección de anomalías en **Ring 0**.

1.3 Por qué un driver malicioso es peligroso

Un driver es un componente de software diseñado para permitir la comunicación entre el sistema operativo y un dispositivo, servicio o componente concreto del sistema. En Windows, muchos drivers se ejecutan en modo kernel, el nivel de privilegio más alto del sistema. Esto les permite **interactuar directamente con recursos críticos** como la memoria, los dispositivos hardware, las interrupciones, las estructuras internas del kernel y otros componentes esenciales del sistema operativo.

Esta posición privilegiada hace que los drivers sean elementos especialmente sensibles desde el punto de vista de la seguridad. Un driver legítimo necesita estos privilegios para realizar tareas de bajo nivel, pero un **driver malicioso puede abusar de ese mismo nivel de acceso para comprometer el sistema de forma profunda**. A diferencia de una aplicación ejecutada en modo usuario, un driver en modo kernel puede operar con muchas menos restricciones y con una visibilidad mucho mayor sobre el funcionamiento interno de Windows.

La peligrosidad de un driver malicioso se debe, en primer lugar, a **su capacidad para acceder y modificar memoria del kernel**. Desde Ring 0, un atacante podría alterar estructuras internas del sistema operativo, modificar punteros, cambiar permisos, manipular listas enlazadas o interferir en el funcionamiento normal de procesos y servicios. Este tipo de acciones puede permitir **ocultar procesos, modificar información del sistema, alterar el comportamiento de llamadas al sistema o dificultar la detección por parte de herramientas de seguridad**.

Otra característica peligrosa es su capacidad para ocultar actividad maliciosa. Muchos rootkits utilizan drivers para **manipular estructuras del kernel y esconder procesos, ficheros, conexiones de red o claves del registro**. Por ejemplo, mediante técnicas como **DKOM**, un driver podría modificar estructuras asociadas a procesos para que un proceso malicioso no aparezca en listados convencionales. De forma similar, mediante técnicas de hooking, podría interceptar funciones del sistema para alterar la información que recibe una herramienta de análisis o monitorización.

Además, un driver malicioso puede utilizarse para **desactivar o debilitar mecanismos de protección del sistema operativo**. Windows incorpora diferentes medidas de seguridad para proteger procesos críticos, controlar la carga de drivers, impedir modificaciones no autorizadas del kernel o verificar la integridad de determinados componentes. Sin embargo, si un atacante consigue ejecutar código en modo kernel, puede intentar manipular estas protecciones o buscar formas de evadirlas. Esto convierte a los drivers maliciosos en una herramienta especialmente útil para amenazas avanzadas.

También es importante destacar que un driver malicioso **puede servir como puente entre un malware en modo usuario y el kernel**. Un proceso malicioso ejecutado en **Ring 3** puede tener limitaciones importantes, pero si consigue comunicarse con un driver en **Ring 0**, puede delegar en él las operaciones más sensibles. De esta forma, el driver actúa como un componente privilegiado que permite ejecutar acciones que normalmente estarían fuera del alcance de una aplicación convencional.

Desde el punto de vista de la estabilidad, **los drivers también representan un riesgo importante**. Un error en modo usuario suele provocar el cierre de una aplicación concreta, pero un error en modo kernel puede comprometer todo el sistema. Una escritura incorrecta en memoria, un puntero mal gestionado o una modificación inadecuada de una estructura crítica puede provocar una caída completa del sistema mediante una pantalla azul o BSOD.

1.4 Por qué WinDbg es útil para inspeccionar el kernel

WinDbg es una herramienta de depuración desarrollada por Microsoft que permite analizar en profundidad el comportamiento interno de Windows. Aunque puede utilizarse para depurar aplicaciones en modo usuario, su utilidad resulta especialmente relevante en el análisis del kernel, ya que permite acceder a información que normalmente no está disponible desde herramientas convencionales ejecutadas en **Ring 3**.

En el contexto del análisis de malware y reversing de sistemas Windows, **WinDbg permite inspeccionar el estado interno del sistema operativo, analizar módulos cargados, revisar procesos e hilos, consultar estructuras del kernel, examinar memoria y estudiar el comportamiento de drivers**. Esta capacidad lo convierte en una herramienta especialmente útil para detectar anomalías asociadas a código ejecutado en **Ring 0**.

Una de las principales ventajas de WinDbg es que permite trabajar directamente con **símbolos del sistema**. **Los símbolos proporcionan información sobre nombres de funciones, variables, estructuras y módulos, lo que facilita la interpretación de los datos internos del sistema operativo**. Sin una correcta configuración de símbolos, el análisis del kernel se vuelve mucho más complejo, ya que las direcciones de memoria y las funciones no pueden identificarse de forma clara.

Gracias a esta información, se puede **utilizar comandos de WinDbg** para examinar estructuras críticas del kernel, como objetos de driver, procesos, hilos, módulos cargados o regiones de memoria. Por ejemplo, es posible listar los drivers presentes en el sistema, inspeccionar un **DRIVER_OBJECT**, revisar rutinas de despacho, analizar estructuras como **EPROCESS** o comprobar si determinados punteros apuntan a zonas de memoria esperadas o sospechosas.

Esto es especialmente importante en la detección de malware en modo kernel. **Un rootkit o driver malicioso puede intentar ocultar su presencia** modificando estructuras internas del sistema, interceptando llamadas, alterando punteros o manipulando listas enlazadas. Muchas de estas modificaciones pueden no ser visibles desde herramientas normales de administración o monitorización, pero sí pueden investigarse mediante un depurador de kernel como WinDbg.

WinDbg también resulta útil porque permite **comparar el estado esperado del sistema con el estado real observado en memoria**. Por ejemplo, un driver legítimo debería tener sus rutinas principales dentro del rango de memoria correspondiente a su propio módulo. Si una rutina apunta a una dirección externa o a una región no identificada, podría tratarse de un indicador de **hooking**, manipulación o comportamiento anómalo. Del mismo modo, una estructura del kernel con valores incoherentes puede indicar una modificación provocada por malware o por un driver defectuoso.

Además de la inspección manual, WinDbg permite **automatizar tareas mediante scripting**. Esta característica es fundamental cuando se quieren repetir comprobaciones sobre múltiples estructuras, módulos o direcciones de memoria. En lugar de ejecutar manualmente una larga secuencia de comandos, el analista puede desarrollar scripts que recopilen información, apliquen reglas básicas de detección y muestren resultados de forma más rápida y ordenada.

1.5 Objetivo de este trabajo

El objetivo principal de este trabajo es analizar cómo WinDbg puede utilizarse para inspeccionar el kernel de Windows y detectar posibles anomalías asociadas a código ejecutado en **Ring 0**. Para ello, se toma como caso de estudio **mimidrv.sys**, el driver de Mimikatz, por tratarse de un componente con capacidades sensibles en modo kernel que permite comprender mejor los riesgos asociados a la carga y ejecución de drivers con privilegios elevados.

El trabajo no tiene como finalidad ejecutar malware real ni comprometer un sistema, sino estudiar desde una perspectiva defensiva qué tipo de elementos pueden analizarse cuando existe un driver con capacidades en **Ring 0**. A partir de este caso de estudio, se pretende mostrar cómo se puede utilizar WinDbg para revisar módulos cargados, objetos de driver, estructuras internas del kernel, direcciones de memoria y rutinas asociadas a un controlador.

De forma concreta, se busca cumplir los siguientes objetivos:

- Comprender la importancia de Ring 0 dentro de la arquitectura de Windows.
 - Explicar por qué los drivers de kernel pueden suponer un riesgo de seguridad cuando son maliciosos, vulnerables o abusados por un atacante.
 - Utilizar WinDbg para realizar una inspección manual de drivers y estructuras del kernel.
 - Identificar posibles indicadores de anomalía, como drivers sospechosos, punteros fuera de rango, rutinas de despacho modificadas o estructuras incoherentes.
 - Documentar los comandos utilizados, las capturas obtenidas y la interpretación de los resultados.
 - Desarrollar un pequeño script que automatice algunas comprobaciones básicas sobre drivers o estructuras del kernel.
 - Valorar las ventajas y limitaciones de este tipo de análisis frente a herramientas ejecutadas únicamente en modo usuario.
-

2. Fundamento teórico

Antes de abordar la parte práctica del análisis con WinDbg, es necesario introducir algunos conceptos fundamentales relacionados con el funcionamiento interno de Windows en modo kernel.

Windows está diseñado como un sistema operativo dividido en diferentes capas y niveles de privilegio. En la parte menos privilegiada se encuentra el modo usuario, donde se ejecutan las aplicaciones convencionales. En la parte más privilegiada se encuentra el modo kernel, donde operan componentes críticos como el propio kernel, el gestor de memoria, el planificador, la capa de abstracción de hardware y los drivers.

Esta separación es esencial para la estabilidad y seguridad del sistema. Sin embargo, también implica que cualquier componente que consiga ejecutarse en modo kernel obtiene una posición especialmente sensible. Desde ese nivel es posible acceder a memoria compartida del sistema, interactuar con estructuras internas, gestionar dispositivos, modificar punteros, registrar rutinas de control y alterar el comportamiento normal del sistema operativo.

Por este motivo, los drivers de kernel tienen una importancia especial en el análisis de malware avanzado. Aunque la mayoría de drivers son legítimos y necesarios para el funcionamiento del sistema, también pueden ser utilizados de forma maliciosa o abusados por atacantes. Un driver vulnerable, manipulado o desarrollado con fines ofensivos puede proporcionar a un malware capacidades que no tendría ejecutándose únicamente en modo usuario.

2.1 Qué es un driver de kernel

Un driver de kernel es un componente de software que se ejecuta en modo kernel y cuya función principal es permitir que el sistema operativo interactúe con dispositivos hardware, recursos internos o servicios de bajo nivel. En Windows, los drivers actúan como **intermediarios entre el sistema operativo y determinados componentes del sistema, proporcionando una interfaz controlada para realizar operaciones que requieren privilegios elevados.**

Aunque tradicionalmente se asocia el concepto de driver con dispositivos físicos, como tarjetas de red, discos, teclados, ratones o tarjetas gráficas, en Windows también existen drivers que no están vinculados directamente a un dispositivo hardware concreto. Algunos drivers se utilizan para **implementar funcionalidades internas, mecanismos de seguridad, filtros del sistema de archivos, soluciones antivirus, herramientas de virtualización o componentes de monitorización.**

La característica más importante de un driver de kernel es que **se ejecuta en Ring 0**. Esto significa que tiene un nivel de privilegio muy superior al de una aplicación normal en modo usuario. Desde este nivel, el driver puede acceder a recursos críticos del sistema, interactuar con memoria del kernel, comunicarse con otros componentes internos, registrar rutinas de entrada/salida y responder a peticiones enviadas desde procesos en modo usuario.

En Windows, **cuando un driver se carga en memoria, el sistema operativo crea y mantiene una serie de estructuras internas asociadas a él.** Una de las más relevantes es **DRIVER_OBJECT**, que representa al driver dentro del kernel. Esta estructura contiene información importante, como el nombre del driver, su dirección en memoria, el dispositivo asociado y las rutinas que el driver expone para gestionar determinadas operaciones.

Entre esas rutinas se encuentran las llamadas **dispatch routines**. Estas funciones permiten que el driver responda a peticiones del sistema, como operaciones de lectura, escritura, creación, cierre o control de entrada/salida. Desde el punto de vista del análisis de seguridad, estas rutinas son especialmente interesantes porque indican qué código se ejecutará cuando el driver reciba determinadas solicitudes.

Un driver legítimo debería tener sus rutinas y punteros apuntando a zonas de memoria coherentes con el propio módulo cargado. Si durante el análisis se observa que una rutina apunta a una dirección inesperada, a una región de memoria no identificada o a código perteneciente a otro módulo, podría tratarse de un indicio de manipulación, **hooking** o comportamiento sospechoso.

Los drivers de kernel también pueden comunicarse con procesos en modo usuario. Para ello, suelen utilizar mecanismos como los códigos de control de entrada/salida, conocidos como **IOCTL**. Este mecanismo permite que una aplicación en modo usuario envíe peticiones al driver para solicitar operaciones concretas. En un contexto legítimo, esta comunicación permite gestionar dispositivos o servicios. En un contexto malicioso, puede utilizarse para que un malware en modo usuario delegue acciones privilegiadas en un driver que opera en Ring 0.

El análisis de drivers de kernel resulta, por tanto, esencial en la detección de anomalías en Ring 0.

2.2 Qué es un **DRIVER_OBJECT**

Un **DRIVER_OBJECT** es una **estructura interna del kernel de Windows que representa a un driver cargado en el sistema.** Cuando un controlador de modo kernel se carga correctamente, Windows crea un objeto asociado que contiene información relevante sobre ese driver y sobre las rutinas que utilizará para gestionar las solicitudes que reciba.

Esta estructura es especialmente importante porque actúa como punto de referencia para el sistema operativo a la hora de interactuar con el driver. **A través del DRIVER_OBJECT, Windows puede localizar el código del controlador, conocer sus funciones principales, gestionar su descarga y dirigir hacia él las peticiones de entrada/salida correspondientes.** Permite observar cómo está registrado un driver dentro del kernel. Al inspeccionarlo, obtenemos información sobre la dirección base del driver, su función de descarga, los dispositivos asociados y la tabla de funciones que procesan las operaciones solicitadas al controlador.

Uno de los campos relevantes de esta estructura es **DriverStart**, **que apunta a la dirección de inicio del código ejecutable del driver en memoria.** Este campo permite ubicar dónde comienza el módulo cargado y puede utilizarse como referencia para comprobar si determinadas funciones o punteros pertenecen realmente al rango de memoria del driver analizado.

Otro campo importante es **DriverUnload**, que contiene la dirección de la función que se ejecuta cuando el driver deja de ser necesario y el sistema procede a descargarlo de memoria. Aunque no todos los drivers implementan una rutina de descarga, su presencia o ausencia puede aportar información útil durante el análisis. En algunos casos, un driver malicioso podría no implementar correctamente esta función para dificultar su eliminación o análisis.

Uno de los elementos más interesantes desde el punto de vista de la detección es **MajorFunction**. Este campo es un vector de punteros a funciones, donde cada entrada corresponde a una rutina encargada de procesar un tipo concreto de solicitud de entrada/salida. Estas funciones se conocen como **dispatch routines** y se encargan de gestionar operaciones como apertura, cierre, lectura, escritura o control del dispositivo.

Las **dispatch routines** son relevantes porque indican qué código se ejecutará cuando el driver reciba determinadas peticiones. Por ejemplo, cuando una aplicación en modo usuario se comunica con un driver mediante una operación de control, el sistema dirige esa petición hacia la rutina correspondiente registrada en la tabla **MajorFunction**. Por ello, revisar estas direcciones permite entender cómo el driver responde a las solicitudes y qué zonas de memoria ejecuta.

En un driver legítimo, lo esperable es que las rutinas registradas en **MajorFunction** apunten a direcciones coherentes dentro del propio módulo del driver o dentro de componentes legítimos del sistema. Si una de estas entradas apunta a una región de memoria desconocida, a una dirección fuera del rango esperado o a código perteneciente a otro módulo no relacionado, podría tratarse de un indicio de manipulación, hooking o comportamiento anómalo.

Con WinDbg, esta estructura puede inspeccionarse mediante comandos como **!drvobj**, que permite **mostrar información asociada a un driver concreto**. También puede analizarse su definición mediante comandos como **dt nt!_DRIVER_OBJECT**, lo que permite consultar sus campos y comprender cómo Windows organiza internamente la información de los drivers cargados.

2.3 Qué estructuras del kernel vamos a observar

Para realizar una detección básica de anomalías en **Ring 0** no basta con listar procesos o drivers desde herramientas en modo usuario. Es necesario **observar determinadas estructuras internas del kernel** que Windows utiliza para representar procesos, drivers, cadenas, listas enlazadas y otros elementos críticos del sistema. Estas estructuras permiten comprender cómo organiza Windows la información internamente y qué puntos pueden ser manipulados por un driver malicioso o por un rootkit.

En este trabajo se observarán principalmente estructuras relacionadas con drivers y procesos, ya que el objetivo es analizar cómo un componente con capacidades en modo kernel, como **mimidrv.sys**, puede ser inspeccionado mediante WinDbg. Las estructuras que se observarán serán las siguientes:

Estructura o elemento	Utilidad en el análisis
<code>DRIVER_OBJECT</code>	Representa un driver cargado en el kernel y permite revisar sus rutinas principales.
<code>MajorFunction</code>	Tabla de rutinas de despacho asociadas al driver. Es útil para detectar punteros anómalos.
<code>EPROCESS</code>	Representa procesos en el kernel y permite estudiar posibles manipulaciones u ocultación.
<code>LIST_ENTRY</code>	Permite entender cómo Windows enlaza objetos internos mediante listas doblemente enlazadas.
<code>UNICODE_STRING</code>	Representa cadenas Unicode utilizadas en nombres de objetos, drivers y rutas.
Módulos cargados	Permiten identificar rangos de memoria válidos y comprobar si los punteros son coherentes.

2.4 Relación con técnicas de rootkit

La relación entre los drivers de kernel y los rootkits es especialmente importante porque muchos rootkits utilizan drivers como mecanismo para ejecutar código en **Ring 0**. Al cargarse como controladores, estos componentes pueden acceder a memoria del kernel, interceptar funciones críticas, manipular objetos internos y modificar el comportamiento normal del sistema operativo. Esta capacidad les permite realizar acciones que serían mucho más difíciles desde modo usuario.

Una de las técnicas clásicas asociadas a rootkits es el **hooking**. Esta técnica consiste en **interceptar una función o una tabla de funciones para redirigir el flujo de ejecución hacia código controlado por el atacante**. En el contexto del kernel de Windows, esto puede afectar a estructuras críticas como la **SSDT**, la **IDT** o las rutinas internas de determinados drivers. El objetivo suele ser modificar el comportamiento del sistema, ocultar información o controlar determinadas operaciones antes de que lleguen a su destino legítimo.

El **SSDT Hooking**, por ejemplo, se basa en la manipulación de la **System Service Descriptor Table**, una tabla relacionada con las llamadas al sistema. Si un rootkit modifica entradas de esta tabla, puede interceptar llamadas realizadas desde modo usuario hacia el kernel. De esta forma, podría alterar el resultado de funciones utilizadas para enumerar procesos, abrir ficheros, consultar información del sistema o acceder a determinados recursos. Aunque las versiones modernas de Windows incorporan protecciones contra este tipo de modificaciones, sigue siendo una técnica relevante desde el punto de vista histórico y conceptual.

Otra técnica importante es el **IDT Hooking**, que consiste en modificar la **Interrupt Descriptor Table**. La **IDT** contiene información sobre las rutinas que deben ejecutarse cuando se producen interrupciones hardware o software. Si un rootkit altera alguna entrada de esta tabla, podría redirigir la ejecución hacia código malicioso y controlar determinados eventos del sistema. Esta técnica es especialmente sensible porque afecta a mecanismos de bajo nivel del procesador y del sistema operativo.

También destaca la técnica conocida como **DKOM**, o **Direct Kernel Object Manipulation**. A diferencia del hooking, **DKOM** no se basa necesariamente en interceptar funciones, sino en modificar directamente objetos y estructuras internas del kernel. Por ejemplo, un rootkit podría alterar la lista de procesos activos para ocultar un proceso malicioso. El proceso seguiría existiendo y ejecutándose, pero podría dejar de aparecer en determinadas herramientas de enumeración si estas dependen de la estructura manipulada.

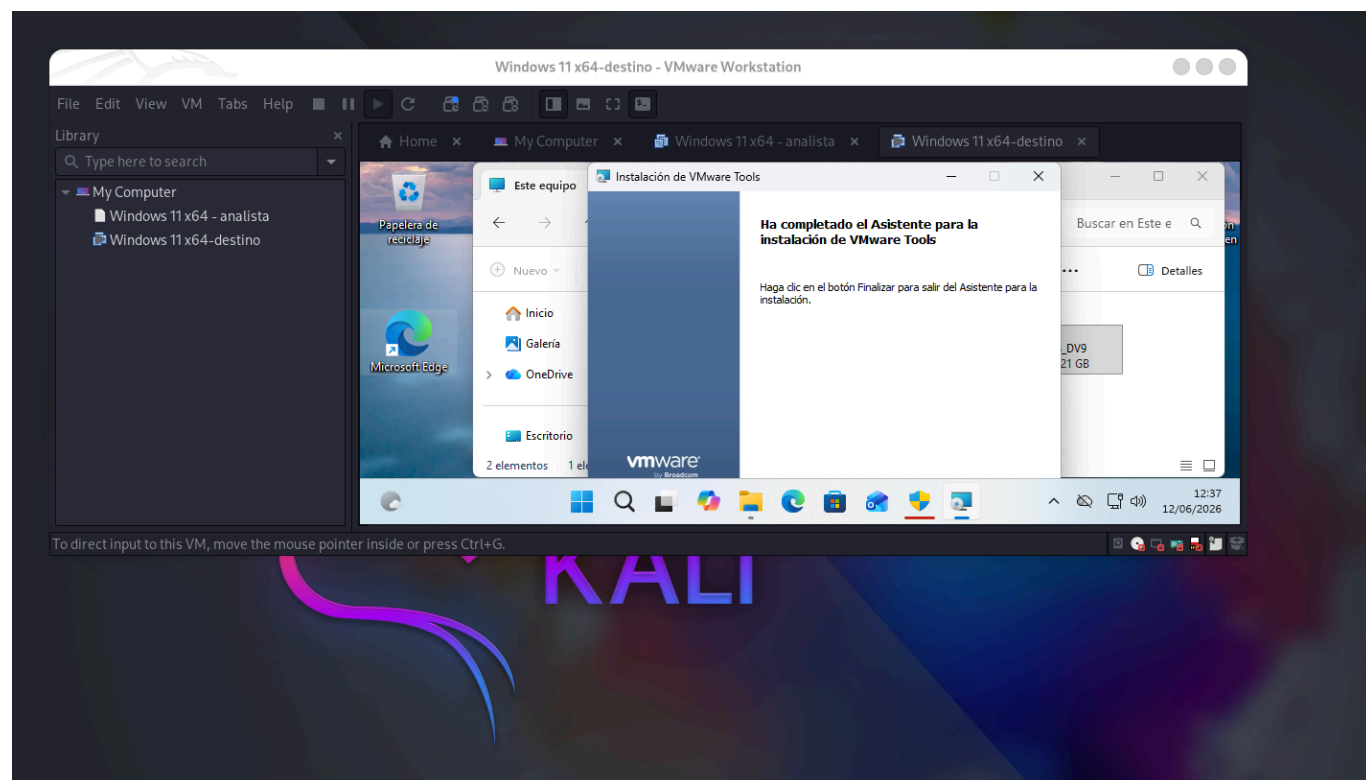
La detección de rootkits en modo kernel es compleja porque muchas de sus modificaciones se producen en **zonas que no son visibles desde herramientas convencionales en modo usuario**. Un proceso oculto mediante **DKOM**, una entrada modificada en una tabla interna o una rutina alterada en un objeto de driver pueden pasar desapercibidas si solo se utilizan herramientas normales del sistema. Por este motivo, el uso de WinDbg resulta especialmente útil, ya que permite inspeccionar directamente memoria, estructuras y objetos internos del kernel.

3. Entorno de laboratorio

El laboratorio se ejecuta sobre un sistema físico Kali Linux que actúa únicamente como plataforma anfitriona de VMware. Para la depuración remota de kernel se emplean dos máquinas virtuales Windows: una máquina analista, donde se instala y ejecuta WinDbg, y una máquina destino con Windows 11 actualizado, cuyo kernel será inspeccionado.

Esquema del laboratorio:

```
Host Kali
├── VMware
│   ├── VM 1: Windows 10/11 analista
│   │   └── WinDbg instalado
│   └── VM 2: Windows 11 destino
│       └── Sistema que se va a depurar
```



3.1 Sistema operativo Windows usado

El sistema operativo utilizado en el laboratorio es Windows 11 completamente actualizado. La elección de Windows 11 permite trabajar sobre un entorno moderno, con mecanismos de protección actuales y una arquitectura representativa de los sistemas Windows utilizados en entornos reales.

Windows 11 incorpora diferentes medidas de seguridad orientadas a proteger el sistema frente a modificaciones no autorizadas, especialmente en componentes críticos como el kernel, los drivers y los procesos protegidos. Entre estas medidas se encuentran la **firma de drivers**, las **protecciones del kernel**,

mecanismos de aislamiento y otras funcionalidades destinadas a reducir el impacto de código malicioso ejecutado con privilegios elevados.

3.2 Máquina host física: Kali Linux con VMware

El laboratorio se ejecuta sobre una máquina física con Kali Linux, que actúa como sistema anfitrión principal para la plataforma de virtualización VMware.

```
└─$ hostnamectl; echo; lsb_release -a; echo; uname -a; echo; lscpu; echo;
free -h; echo; df -h /
  Static hostname: xxxx
        Icon name: computer-desktop
        Chassis: desktop 
        Machine ID: xxxx
        Boot ID: xxxx
  Operating System: Kali GNU/Linux Rolling
        Kernel: Linux 6.19.14+kali-amd64
        Architecture: x86-64
  Hardware Vendor: Gigabyte Technology Co., Ltd.
  Hardware Model: Z790 GAMING X AX
  Hardware Version: x.x
  Firmware Version: F9b
        Firmware Date: Thu 2023-11-09
        Firmware Age: 2y 7month 2d

No LSB modules are available.
Distributor ID: Kali
Description:    Kali GNU/Linux Rolling
Release:       2026.2
Codename:      kali-rolling

Linux xxniwexx 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-
1+kali1 (2026-05-05) x86_64 GNU/Linux

Architecture:            x86_64
  CPU op-mode(s):        32-bit, 64-bit
  Address sizes:          39 bits physical, 48 bits virtual
  Byte Order:             Little Endian
CPU(s):                  32
  On-line CPU(s) list:    0-31
Vendor ID:               GenuineIntel
  Model name:             Intel(R) Core(TM) i9-14900KF
  CPU family:             6
...
...

total      usado      libre  compartido  búf/caché
disponible
Mem:       62Gi      7,3Gi      13Gi      111Mi      43Gi
55Gi
```


Inter:	63Gi	20Ki	63Gi
S.ficheros		Tamaño Usados	Disp Uso% Montado en
/dev/mapper/xxniwexx--vg-root	442G	388G	33G 93% /

3.3 Máquina analista Windows con WinDbg

La máquina analista es una máquina virtual Windows desplegada en VMware cuya función principal es ejecutar WinDbg y conectarse de forma remota a la máquina destino. Esta separación permite que el análisis del kernel se realice desde un sistema externo al que se está depurando, siguiendo un modelo de depuración remota más adecuado para el estudio de componentes en **Ring 0**.

La versión utilizada en el laboratorio es **WinDbg 1.2603.20001.0**, instalada en el sistema Windows 11 de la máquina virtual. [Enlace a WinDbg](#).

La máquina analista se encuentra conectada a la misma red virtual que la máquina destino mediante una red de tipo host-only configurada en VMware. Esta configuración permite que ambas máquinas se comuniquen entre sí para establecer la sesión de depuración remota, pero evita exponer el laboratorio directamente a redes externas.

Una vez configurada la máquina analista, desde WinDbg se podrán ejecutar comandos para listar módulos cargados, consultar drivers, inspeccionar estructuras como DRIVER_OBJECT o EPROCESS, revisar direcciones de memoria y ejecutar scripts de automatización. Esta máquina será, por tanto, el punto central desde el que se realizará tanto el análisis manual como la ejecución del script desarrollado en la práctica.

El uso de una máquina analista independiente aporta mayor control durante la depuración. Si la máquina destino queda detenida, genera un error o sufre una caída del sistema, la máquina analista permanece operativa y conserva la sesión de análisis. Esto facilita la investigación de fallos, la captura de evidencias y la documentación del proceso.

Adaptadores de red en la MV analista:

- Adaptador 1: Host-only / red interna → para hablar con la destino 192.168.75.129
- Adaptador 2: NAT → solo para descarga de símbolos

Destino:

- Adaptador 1: Host-only / misma red interna → 192.168.75.129

3.3.1 Carga de los símbolos

Una parte esencial de la configuración de WinDbg es la **carga correcta de símbolos**. Los símbolos permiten que el depurador traduzca direcciones de memoria en nombres de funciones, estructuras y variables reconocibles. Sin esta información, el análisis del kernel sería mucho más complejo, ya que muchas direcciones aparecerían únicamente como valores numéricos sin contexto suficiente.

Para configurar los símbolos se puede utilizar el **servidor público de símbolos de Microsoft**. Durante la práctica, esta configuración permitirá analizar módulos del sistema como `ntoskrnl.exe`, consultar estructuras internas y ejecutar comandos relacionados con drivers y procesos. Una configuración básica de símbolos puede realizarse desde WinDbg mediante comandos como:

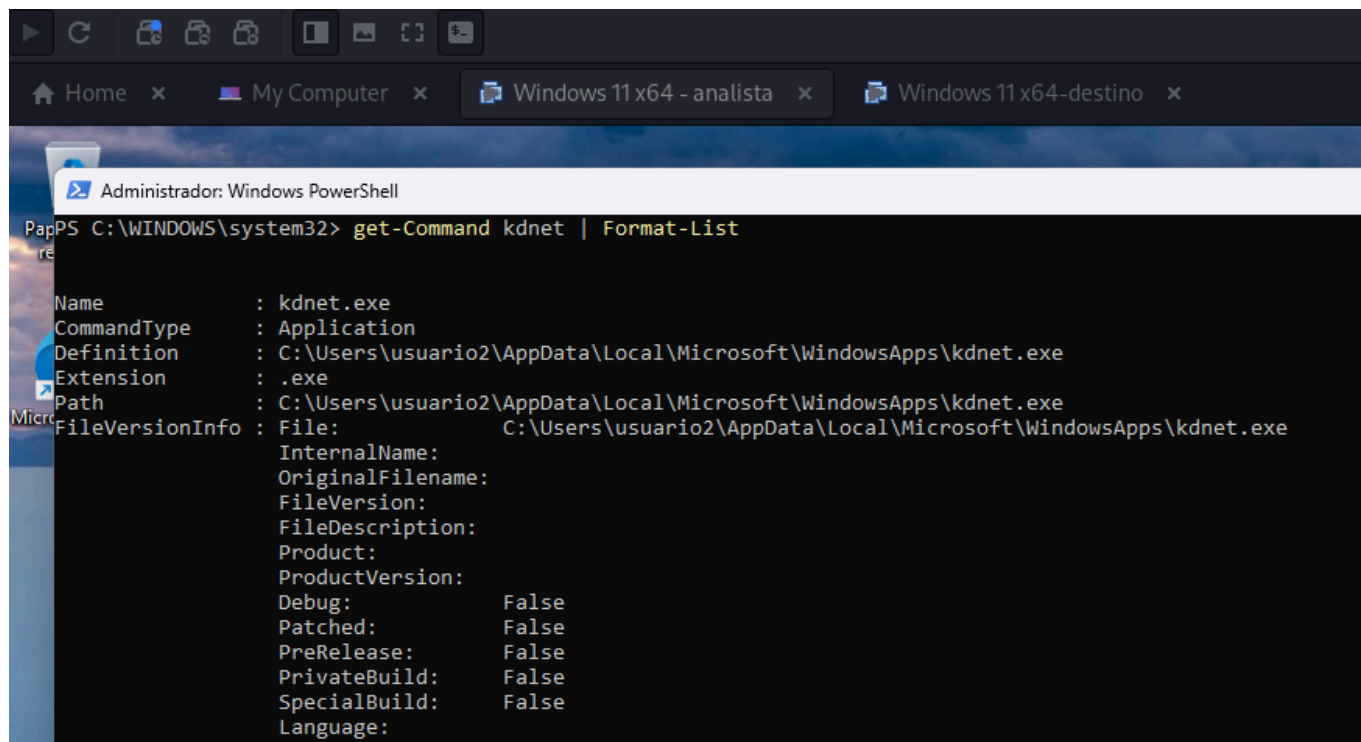
```
.symfix  
.reload
```

También puede establecerse una ruta de símbolos personalizada, por ejemplo:

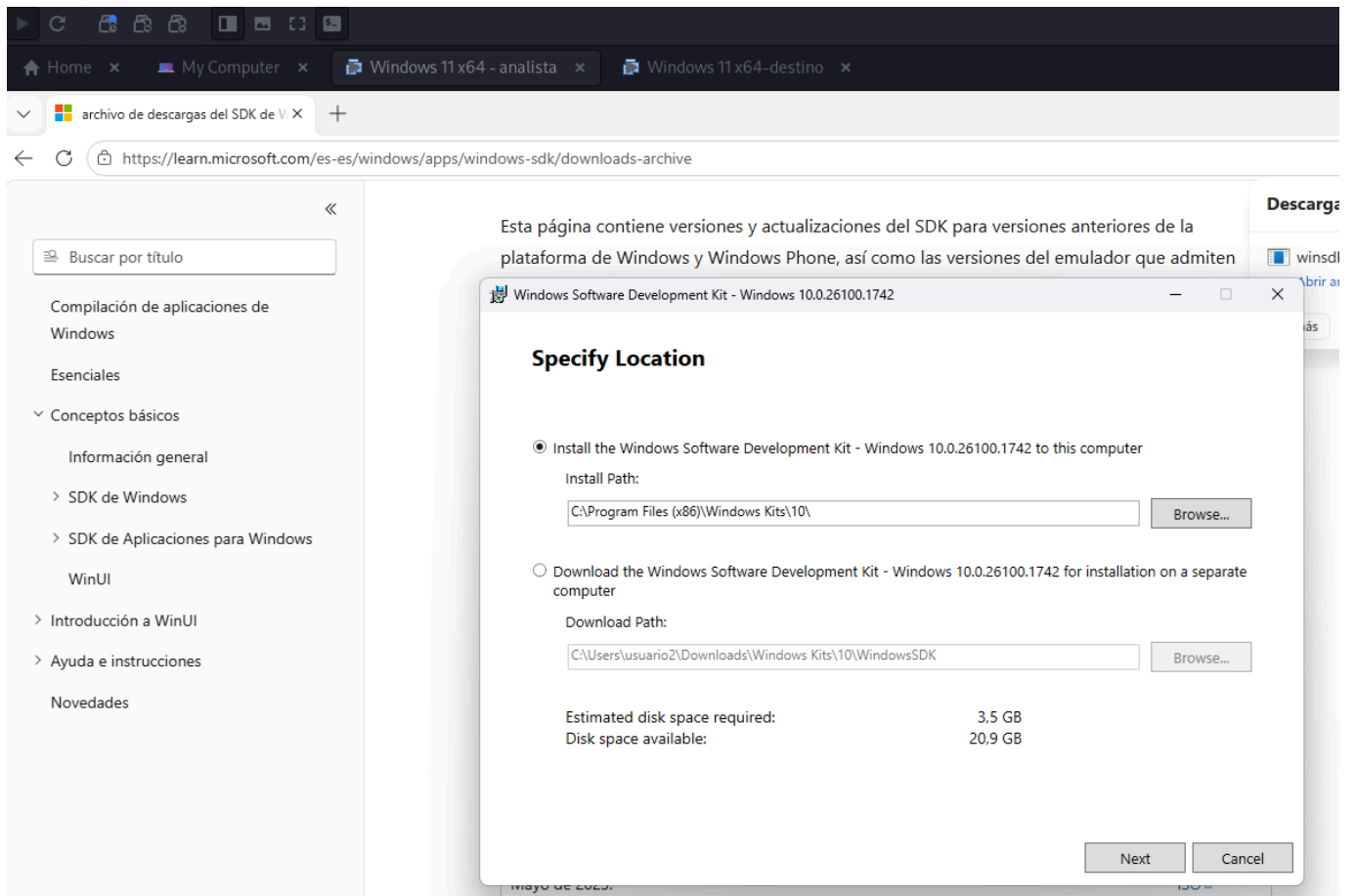
```
.sympath srv*C:\Symbols*https://msdl.microsoft.com/download/symbols  
.reload
```

3.3.2 Localización de kdnet.exe

```
Get-ChildItem -Path "C:\Program Files", "C:\Program Files (x86)" -Recurse -  
Filter VerifiedNICList.xml -ErrorAction SilentlyContinue
```

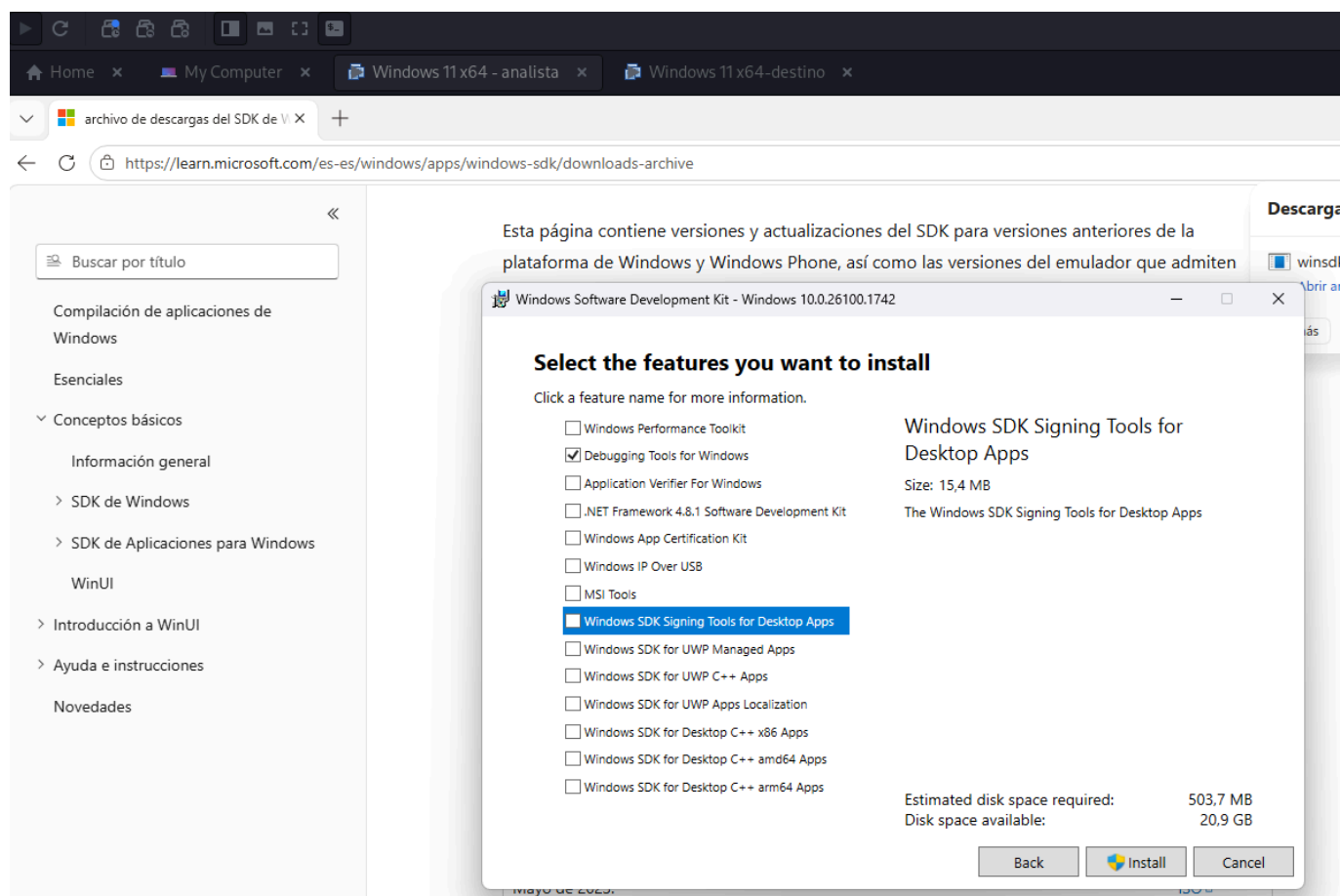


Durante la configuración inicial observamos que **kdnet.exe** aparece en la ruta **C:\Users\usuario2\AppData\Local\Microsoft\WindowsApps**. Sin embargo, dicha ubicación corresponde a los alias de ejecución de aplicaciones instaladas desde **Microsoft Store/WinDbg Preview**, **no a la instalación completa de las herramientas de depuración**. Por ello, es necesario instalar el componente **Debugging Tools for Windows** del **Windows SDK** para obtener los archivos reales **kdnet.exe** y **VerifiedNICList.xml**, necesarios para configurar la depuración remota de kernel en la máquina destino.



Dejamos marcado únicamente:

☒ Debugging Tools for Windows



Cuando termina la instalación, comprobamos que existen:

```
Test-Path "C:\Program Files (x86)\Windows Kits\10\Debuggers\x64\kdnet.exe"
True
```

```
Test-Path "C:\Program Files (x86)\Windows Kits\10\Debuggers\x64\VerifiedNICList.xml"
True
```

Después copiamos estos dos archivos a la máquina destino:

```
kdnet.exe
VerifiedNICList.xml
```

3.4 Máquina destino Windows 11 actualizada

La máquina destino es una máquina virtual con Windows 11 completamente actualizado, desplegada en VMware, sobre la que se realizará el análisis del kernel. Esta máquina actúa como sistema objetivo de la depuración remota y será inspeccionada desde la máquina analista mediante WinDbg.

En el laboratorio, la máquina destino representa el sistema Windows que se desea analizar. Sobre ella se observarán los drivers cargados, los módulos del sistema, las estructuras internas del kernel y los posibles indicadores de anomalía asociados a componentes ejecutados en **Ring 0**. Al tratarse de una máquina virtual, el análisis puede realizarse de forma controlada, reproducible y aislada del sistema físico.

La máquina Windows 11 destino se utilizará para ejecutar las comprobaciones necesarias sobre el estado del kernel. Desde la máquina analista se podrán listar módulos cargados, inspeccionar objetos de driver, revisar estructuras como **DRIVER_OBJECT** y **EPROCESS**, y comprobar direcciones de memoria asociadas a rutinas internas del sistema.

3.5 Configuración de red para Remote Kernel Debugging

Para realizar la depuración remota de kernel es necesario que la máquina analista y la máquina destino puedan comunicarse entre sí a través de una red virtual. La máquina analista y la máquina destino se conectan a la misma red virtual para permitir el establecimiento de la sesión de depuración remota.

La red virtual se configura en modo **host-only** dentro de VMware. Esta configuración permite que ambas máquinas Windows se comuniquen entre sí, pero mantiene el laboratorio aislado de redes externas. Este aislamiento resulta recomendable cuando se trabaja con análisis de kernel, drivers y posibles comportamientos maliciosos, ya que reduce la exposición del entorno y permite controlar mejor las comunicaciones entre los sistemas implicados.

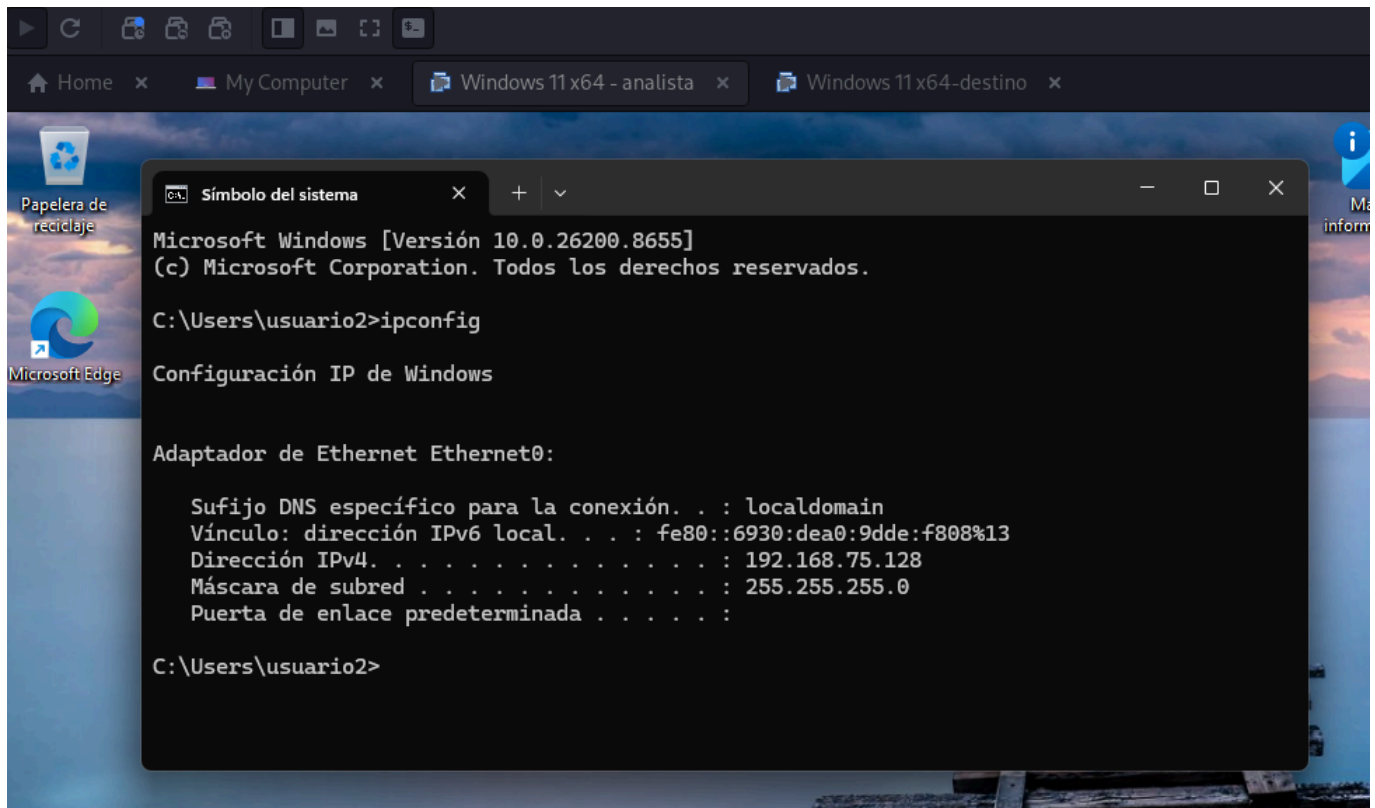
La arquitectura de red utilizada puede representarse de la siguiente forma:

```
Kali Linux físico
├── VMware
│   ├── VM analista: Windows con WinDbg
│   │   └── Adaptador de red: Host-only / red interna
│   └── VM destino: Windows 11 actualizado
│       └── Adaptador de red: Host-only / red interna
```

Antes de configurar WinDbg, es necesario comprobar que ambas máquinas se encuentran en la misma red y que existe conectividad entre ellas. Para ello, en la máquina analista y en la máquina destino se puede consultar la configuración IP mediante:

Verificamos ip MV analista:

```
ipconfig
```

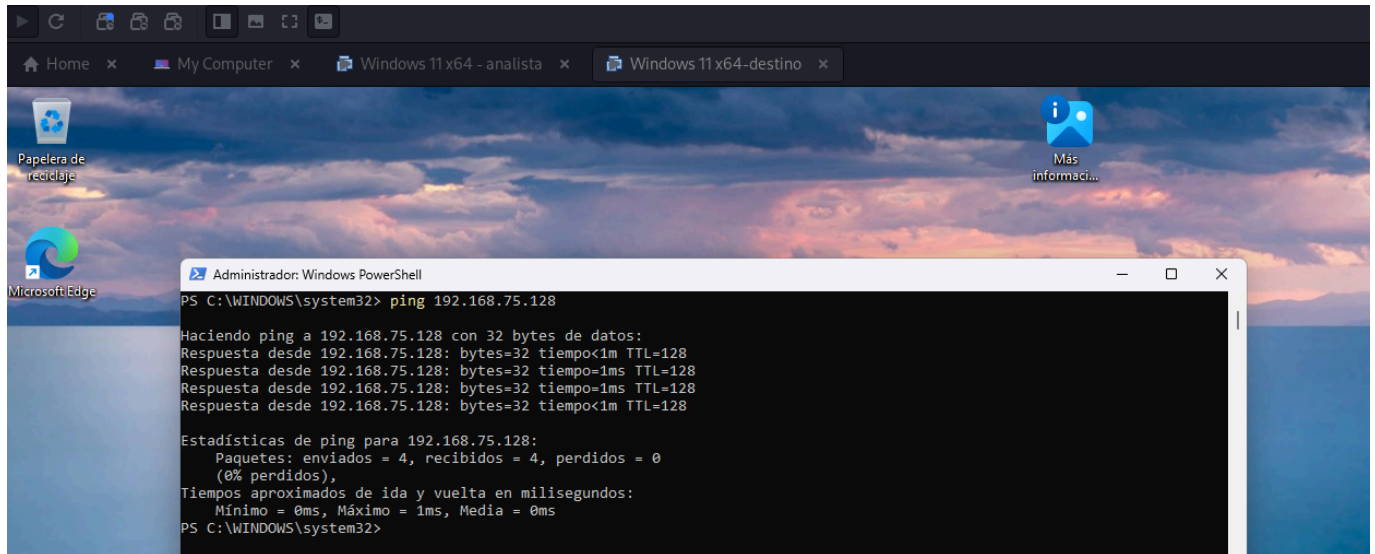


Deshabilitamos el bloqueo ICMP del firewall de Windows: En ambas máquinas virtuales, abrimos Powershell en modo administrador y ejecutamos el siguiente comando para desbloquear ICMP:

```
New-NetFirewallRule -DisplayName "Allow ICMPv4-In" -Protocol ICMPv4 -  
IcmpType 8 -Direction Inbound -Action Allow -Profile Any
```

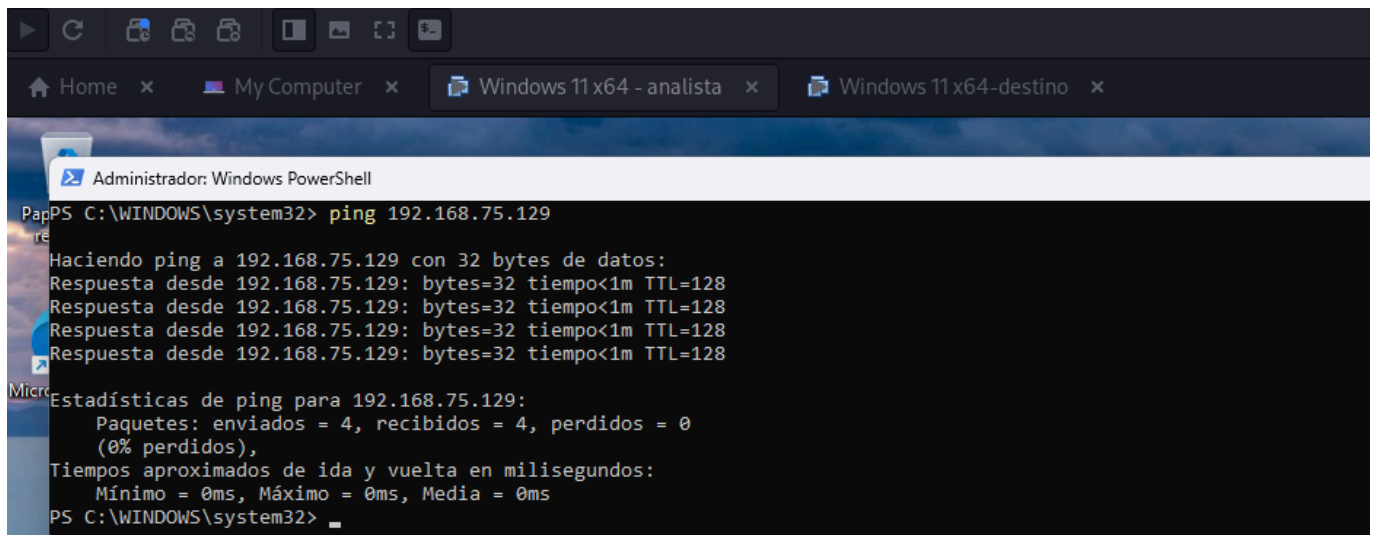
Desde la máquina destino comprobamos la conectividad hacia la máquina analista mediante:

```
ping <IP_MAQUINA_ANALISTA>
```



Y desde la máquina analista hacia la máquina destino mediante:

```
ping <IP_MAQUINA_DESTINO>
```



3.6 Configuración de una conexión de depuración remota

Una vez verificada la conectividad entre la máquina analista y la máquina destino, procedemos a configurar la depuración remota de kernel. En este laboratorio se utiliza una conexión de depuración por red, conocida como **Remote Kernel Debugging over NET**, ya que permite analizar el kernel de la máquina destino desde un sistema independiente donde se ejecuta WinDbg.

La máquina analista es el sistema Windows donde se encuentra instalado WinDbg y tiene asignada la dirección IP **192.168.75.128**. La máquina destino es el sistema Windows 11 actualizado cuyo kernel será depurado y tiene asignada la dirección IP **192.168.75.129**. Ambas máquinas se encuentran dentro de la misma red virtual de VMware, lo que permite establecer comunicación directa entre ellas.

La conexión de depuración se configura desde la máquina destino utilizando la herramienta **kdnet.exe**. Esta utilidad permite habilitar la depuración de kernel por red e indicar la dirección IP de la máquina donde se ejecutará WinDbg, junto con el puerto que se utilizará para establecer la conexión.

En la MV analista, accedemos a la carpeta:

```
C:\Program Files (x86)\Windows Kits\10\Debuggers\x64
```

Se copian los archivos **kdnet.exe y **VerifiedNICList.xml** a una carpeta de trabajo en la máquina destino:**

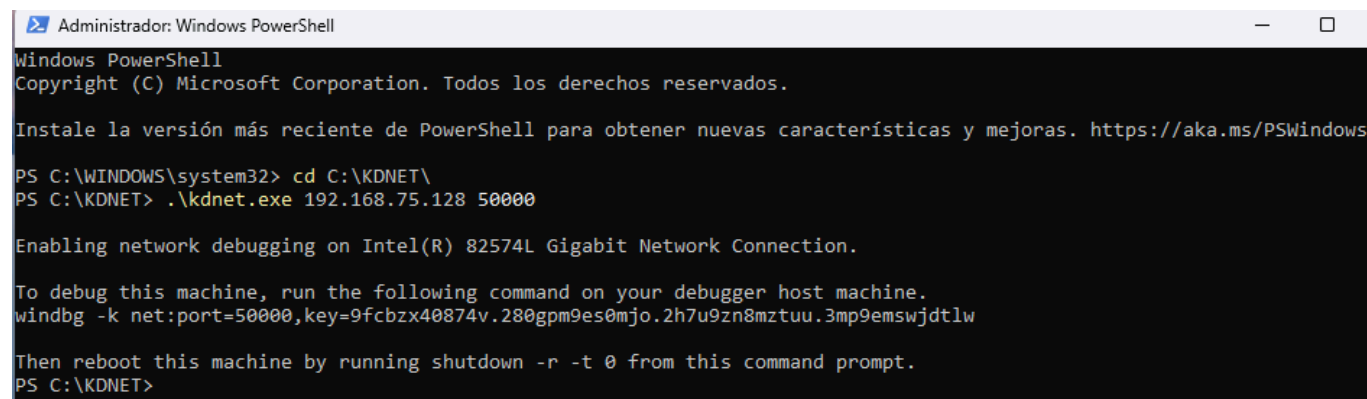
```
C:\KDNET
```

Se abre una consola de comandos con permisos de administrador en la máquina destino y se accede al directorio donde se encuentra **kdnet.exe:**

```
cd C:\KDNET
```

Se ejecuta **kdnet.exe** indicando la dirección IP de la máquina analista y el puerto elegido para la conexión. En este laboratorio se utiliza el puerto **50000**:

```
kdnet.exe 192.168.75.128 50000
```



```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd C:\KDNET\
PS C:\KDNET> .\kdnet.exe 192.168.75.128 50000

Enabling network debugging on Intel(R) 82574L Gigabit Network Connection.

To debug this machine, run the following command on your debugger host machine.
windbg -k net:port=50000,key=9fcbzx40874v.280gpm9es0mjo.2h7u9zn8mztuu.3mp9emswjdtlw

Then reboot this machine by running shutdown -r -t 0 from this command prompt.
PS C:\KDNET>
```

Este comando configura la máquina destino para aceptar una conexión de depuración remota desde la máquina analista. Además, **kdnet.exe** genera una clave de conexión que deberá utilizarse posteriormente en WinDbg. Esta clave es necesaria para autenticar la sesión de depuración entre ambas máquinas, por lo que debemos copiarla y guardarla junto con el puerto utilizado.

De esta forma no tenemos que ejecutar manualmente **bcdedit /debug on**, porque al ejecutar **kdnet.exe** en la máquina destino, la herramienta ya ha configurado la depuración de kernel por red. Así que **kdnet.exe** configura la depuración de red correctamente y además, genera el comando para WinDbg:

```
windbg -k net:port=50000,key=...
```

Por tanto, NO es necesario ejecutar aparte:

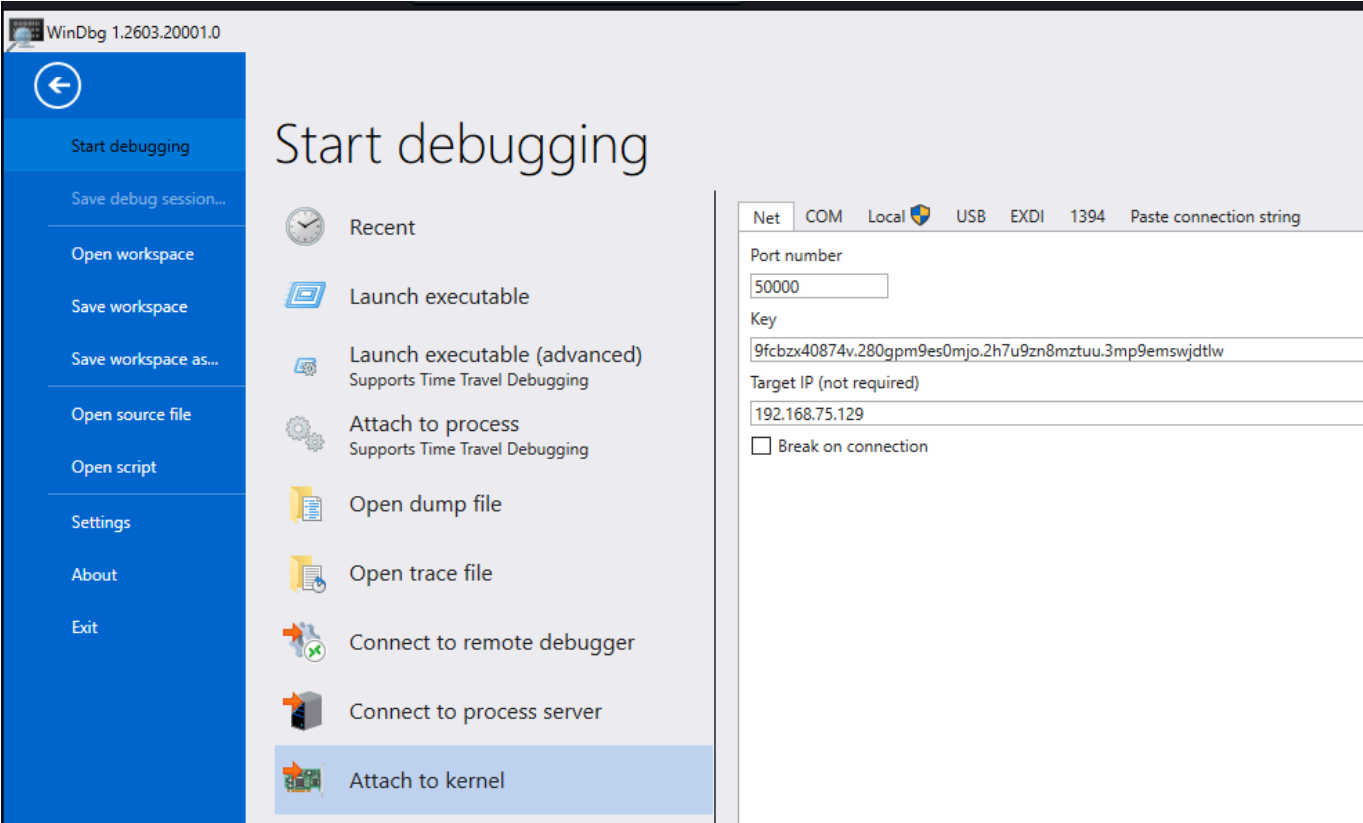
```
bcdedit /debug on
```

Después de realizar la configuración, es necesario reiniciar la máquina destino para que los cambios en la configuración de arranque se apliquen correctamente:

```
shutdown -r -t 0
```

Una vez reiniciada la máquina destino, abrimos WinDbg en la máquina analista. Desde **WinDbg** seleccionamos la opción de conexión al kernel por red:

```
File → Attach to kernel → NET
```



En la ventana de configuración de la conexión se introducen los datos generados durante la configuración con `kdnet.exe`:

Parámetro	Valor utilizado en el laboratorio
Máquina analista	192.168.75.128
Máquina destino	192.168.75.129
Puerto	50000
Clave	Clave generada por <code>kdnet.exe</code>

KD 'net:port=50000,key=****,target=192.168.75.129', Default Connection - WinDbg 1.2603.20001.0

Archivo Home View Breakpoints Time Travel Model Scripting Source Memory Extensions

Break Go Step Out Step Out Back Step Into Step Into Back Step Over Step Over Back Restart Stop Debugging Detach Settings Source Assembly Local Help Feedback

Flow Control Reverse Flow Control End Preferences Help

Disassembly Registers Memory 0

Command X

```
ExtensionRepository : Implicit
UseExperimentalFeatureForNuGetShare : true
AllowNuGetExeUpdate : true
NonInteractiveNuget : true
AllowNuGetMSCredentialProviderInstall : true
AllowParallelInitializationOfLocalRepositories : true
EnableRedirectToChakraJsProvider : false

-- Configuring repositories
----> Repository : LocalInstalled, Enabled: true
----> Repository : UserExtensions, Enabled: true

>>>>>>>>>>>> Preparing the environment for Debugger Extensions Gallery repositories completed, duration 0.016 seconds

***** Waiting for Debugger Extensions Gallery to Initialize *****

>>>>>>>>>>>> Waiting for Debugger Extensions Gallery to Initialize completed, duration 0.031 seconds
----> Repository : UserExtensions, Enabled: true, Packages count: 0
----> Repository : LocalInstalled, Enabled: true, Packages count: 46

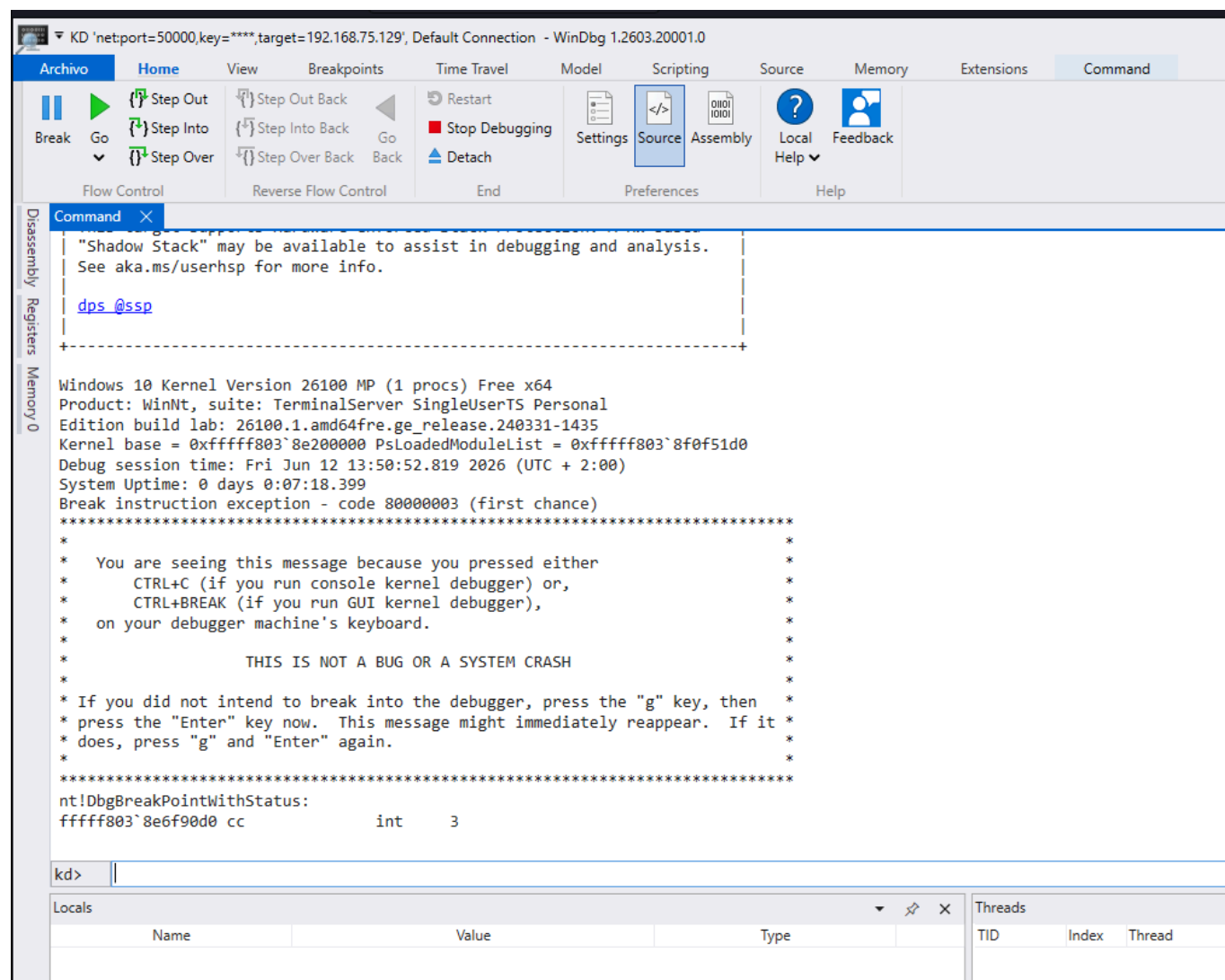
Microsoft (R) Windows Debugger Version 10.0.29547.1002 AMD64
Copyright (c) Microsoft Corporation. All rights reserved.

Using NET for debugging
Opened WinSock 2.0
Using IPv4 only.
Waiting to reconnect...
Connected to target 192.168.75.129 on port 50000 on local IP 192.168.75.128.
You can get the target MAC address by running .kdtargetmac command.
```

BUSY Debuggee is running...

Locals			Threads
Name	Value	Type	

Pulsamos el boton de **Break**:



Como la conexión es correcta, WinDbg nos pasa el sistema destino y nos muestra un **prompt** de comandos:

```
kd>
```

4. Detección manual con WinDbg

Una vez configurado el entorno de laboratorio, establecida la conexión de depuración remota, procedemos a realizar la fase de detección manual con WinDbg. El objetivo de este apartado es mostrar cómo puede utilizarse el depurador para inspeccionar el estado interno de la máquina destino y analizar elementos relacionados con la ejecución de código en **Ring 0**.

La detección manual se centra principalmente en el análisis de drivers cargados, objetos del kernel y estructuras internas asociadas a controladores. Esta fase permite comprender qué información puede obtenerse directamente desde el kernel y qué tipo de indicadores podrían considerarse sospechosos en un escenario de análisis de malware o rootkits.

Para ello, se utilizarán comandos de WinDbg orientados a listar módulos cargados, consultar estructuras internas, enumerar objetos de tipo driver e inspeccionar el DRIVER_OBJECT asociado a un controlador concreto. A partir de esta información se analizarán campos relevantes como DriverStart, DriverSize, DriverUnload y la tabla de rutinas MajorFunction.

4.1. Configuración de símbolos

Una vez establecida correctamente la conexión de depuración remota y obtenido el prompt **kd>** en WinDbg, el siguiente paso consiste en configurar los símbolos. Esta configuración es fundamental para poder interpretar correctamente las direcciones de memoria, funciones, módulos y estructuras internas del kernel de Windows.

Los símbolos permiten que WinDbg traduzca direcciones de memoria en nombres comprensibles de funciones, estructuras, variables y módulos. Sin una configuración adecuada, el análisis del kernel sería mucho más complejo, ya que muchos elementos aparecerían únicamente como direcciones hexadecimales sin contexto suficiente. En una práctica orientada al análisis de drivers y estructuras internas, disponer de símbolos correctamente cargados resulta imprescindible.

En este laboratorio se utiliza el servidor público de símbolos de Microsoft, junto con una carpeta local de caché donde se almacenarán los símbolos descargados. Para ello, en la máquina analista se crea una carpeta local, por ejemplo:

```
C:\Symbols
```

A continuación, desde el prompt **kd>** de WinDbg se configura la ruta de símbolos mediante el siguiente comando:

```
.sympath srv*C:\Symbols*https://msdl.microsoft.com/download/symbols
```

Con este comando se indica a WinDbg que utilice `C:\Symbols` como caché local y que descargue los símbolos necesarios desde el servidor público de Microsoft. Esta configuración permite reutilizar los símbolos descargados en sesiones posteriores, reduciendo el tiempo de carga y evitando descargas repetidas.

Después de configurar la ruta de símbolos, se fuerza la recarga de los símbolos mediante:

```
.reload /f
```

El parámetro `/f` fuerza la recarga de los módulos y símbolos, asegurando que WinDbg intente resolver correctamente la información asociada al sistema destino. Este paso puede tardar unos minutos, especialmente la primera vez, ya que se descargarán símbolos desde Internet.

Para comprobar la ruta de símbolos configurada, ejecutamos:

```
.sympath
```

WinDbg mostrará la ruta activa de símbolos:

```
condrv          The system cannot find the file specified
Ks1D            The system cannot find the file specified
WdNisDrv        The system cannot find the file specified

You can troubleshoot most symbol related issues by turning on symbol loading diagnostics (!sym noisy) and repeating the command that caused symbols to be loaded.
You should also verify that your symbol search path (.sympath) is correct.
kd> .sympath
Symbol search path is: srv*C:\symbols*https://msdl.microsoft.com/download/symbols
Expanded Symbol search path is: srv*C:\symbols*https://msdl.microsoft.com/download/symbols

***** Path validation summary *****
Response      Time (ms)    Location
Deferred                               srv*C:\symbols*https://msdl.microsoft.com/download/symbols

kd> |
```

Locals				Threads			
Name	Value	Type		TID	Index	Thread	Description

4.2 Mostramos información básica del sistema que estamos depurando

El comando `vertarget` se utiliza para verificar la información básica del sistema destino en una sesión de depuración. En esta práctica permite confirmar que WinDbg está conectado correctamente al kernel de la máquina Windows 11 destino, mostrando la versión del kernel, la dirección base de carga del kernel, la dirección de `PsLoadedModuleList` y el tiempo de actividad del sistema.

```
kd> vertarget
Windows 10 Kernel Version 26100 MP (1 procs) Free x64
Product: WinNt, suite: TerminalServer SingleUserTS Personal
Edition build lab: 26100.1.amd64fre.ge_release.240331-1435
Kernel base = 0xfffff805`aac00000 PsLoadedModuleList = 0xfffff805`abaf51d0
Debug session time: Fri Jun 12 20:48:09.285 2026 (UTC + 2:00)
System Uptime: 0 days 0:01:03.721

kd> |
```

Elemento	Interpretación
<code>vertarget</code> responde	WinDbg está conectado al kernel de la VM destino
<code>Kernel base</code> aparece	Se puede inspeccionar el kernel en memoria
<code>PsLoadedModuleList</code> aparece	Se puede acceder a información interna de módulos cargados
<code>System Uptime</code> bajo	La máquina destino acaba de arrancar tras configurar KDNET

Esta información verifica que la sesión de depuración remota está activa y que es posible continuar con la inspección de drivers y estructuras internas del sistema operativo.

4.3 Mostramos información detallada del módulo nt

Comprobamos la carga correcta del módulo principal del kernel mediante el comando `lmv m nt`:

```
kd> lm v m nt
Browse full module list
start      end      module name
Target is not responding. Attempting to reconnect...
fffff802`ad000000 fffff802`ae450000 nt (pdb symbols) c:\symbols\ntkrnlmp.pdb\11D7FE79CC245612055503EE86B80E3E1\ntkrnlmp.pdb
Loaded symbol image file: ntkrnlmp.exe
Image path: ntkrnlmp.exe
Image name: ntkrnlmp.exe
Browse all global symbols functions data Symbol Reload
Image was built with /Brepno flag.
Timestamp: 90B083DA (This is a reproducible build file hash, not a timestamp)
Checksum: 00C74D68
ImageSize: 01450000
Mapping Form: Loaded
Translations: 0000.04b0 0000.04e4 0409.04b0 0409.04e4
Information from resource tables:
```

lm

→ list modules / listar módulos cargados

v

→ verbose / mostrar información detallada

m nt

→ filtrar por el módulo llamado nt

El comando `lmv m nt` se utiliza para mostrar información detallada del módulo `nt`, correspondiente al kernel principal de Windows (`ntkrnlmp.exe`). Este comando permite comprobar la dirección base y final del kernel en memoria, el nombre de la imagen cargada y el estado de los símbolos. En esta práctica resulta

especialmente útil para verificar que los símbolos **PDB** del kernel se han cargado correctamente y que WinDbg puede interpretar estructuras internas del sistema operativo.

4.4 Analizamos los drivers cargados

Una vez cargados los símbolos, podemos realizar una primera comprobación listando los módulos cargados en el sistema destino. El comando **lm** muestra los módulos presentes en memoria, como el kernel de Windows, la **HAL** y los drivers cargados. Si los símbolos están correctamente configurados, WinDbg nos muestra información más completa y nos permite trabajar con nombres de módulos y estructuras del sistema:

lm

The screenshot shows the WinDbg interface with the 'lm' command executed in the Command window. The output lists loaded modules with their addresses, sizes, names, and PDB symbols. The 'Locals' window is empty, and the 'Threads' window shows the current thread.

Address	Size	Name	PDB Symbol
fffff802`3e1e0000	fffff802`3e552000	mcupdate_GenuineIntel	(pdb symbols) c:\symbols\mcupdate_GenuineIntel.pdb\c0057a1004c949d8d8f433a658c4
fffff802`3ea30000	fffff802`3ea96000	kd_02_8086	(pdb symbols) c:\symbols\kd_02_8086.pdb\618715fd38bbf6f2cea9e05c6cd0ede71\kd_02_8086.pdb
fffff802`3eaa0000	fffff802`3eae9000	kdcom	(pdb symbols) c:\symbols\kdnet.pdb\792bc75a8a3cc4eb125188a19603d2d21\kdnet.pdb
fffff802`3eb80000	fffff802`3eb8c000	symcryptk	(pdb symbols) c:\symbols\symcryptk.pdb\286fe2601ffcd49b009b59d1c04d28091\symcryptk.pdb
fffff802`3eb90000	fffff802`3ebba000	tm	(pdb symbols) c:\symbols\tm.pdb\6d04646d41ffc8dd847a9568dc7777611\tm.pdb
fffff802`3ebc0000	fffff802`3ec4b000	CLFS	(pdb symbols) c:\symbols\clfs.pdb\4661e5d634ce7e7389c40ed9252f48531\clfs.pdb
fffff802`3ec50000	fffff802`3ed35000	cng	(pdb symbols) c:\symbols\cng.pdb\86a4f8f6f3f7a435e85e8a982a64e361\cng.pdb
fffff802`3ed40000	fffff802`3ed55000	WinAccel	(pdb symbols) c:\symbols\WinAccel.pdb\6b70e0889575e34c2e72d1c3aff60381\WinAccel.pdb
fffff802`3ed60000	fffff802`3ed7b000	PSHED	(pdb symbols) c:\symbols\psched.pdb\1c867dcac215e6f4c7357f9172eabf681\psched.pdb
fffff802`3ed80000	fffff802`3ed8d000	BOOTVID	(pdb symbols) c:\symbols\bootvid.pdb\3af9a474be26c781c8df1f1f8e201f881\bootvid.pdb
fffff802`3ed90000	fffff802`3eea7000	clipsys	(export symbols) clipsys.sys
fffff802`3eeb0000	fffff802`3ef45000	FLTMGR	(pdb symbols) c:\symbols\fltMgr.pdb\97b804d16014979fec8342fd12eaf7c81\fltMgr.pdb
fffff802`3ef50000	fffff802`3ef81000	ksecdd	(pdb symbols) c:\symbols\ksecdd.pdb\051c9e601ceb320c8ba997ad1c86583b1\ksecdd.pdb
fffff802`3ef90000	fffff802`3eff5000	msrpc	(pdb symbols) c:\symbols\msrpc.pdb\918327f3920a566fc2b944247cf16ed081\msrpc.pdb
fffff802`3f000000	fffff802`3f012000	cmimcext	(pdb symbols) c:\symbols\cmimcext.pdb\51ca057522c22248a789b4c072d62aca1\cmimcext.pdb
fffff802`3f020000	fffff802`3f037000	Werkernel	(pdb symbols) c:\symbols\Werkernel.pdb\073c5b798d32ffc6d62a2e43d08ae861\Werkernel.pdb
fffff802`3f040000	fffff802`3f04d000	ntosex	(pdb symbols) c:\symbols\ntosex.pdb\fe00deea7e709d879866efc74b1a8131\ntosex.pdb
fffff802`3f050000	fffff802`3f117000	win32k	(pdb symbols) c:\symbols\win32k.pdb\717fa19e1a2ca41613e0906e7d343dd51\win32k.pdb
fffff802`3f120000	fffff802`3f140000	watchdog	(pdb symbols) c:\symbols\watchdog.pdb\800d5fce1e997878487f824c49650f061\watchdog.pdb
fffff802`3f150000	fffff802`3f644000	dxgkrnl	(pdb symbols) c:\symbols\dxgkrnl.pdb\41a6579366610f51aae463bca8c08531\dxgkrnl.pdb
fffff802`3f650000	fffff802`3f65d000	WMILIB	(pdb symbols) c:\symbols\wmilib.pdb\088bf6f002fde3fdcf1a1700283b0da11\wmilib.pdb
fffff802`3f660000	fffff802`3f777000	CI	(pdb symbols) c:\symbols\ci.pdb\7d3151f59b8c7db3fb152fed5c5118261\ci.pdb
fffff802`3f780000	fffff802`3f7a0000	globmerger	(pdb symbols) c:\symbols\GlobMerger.pdb\5d98a875fe40915bf5ab3d4924ebdd3b1\GlobMerger.pdb
fffff802`3f790000	fffff802`3f810000	WdF01000	(private pdb symbols) c:\symbols\WdF01000.pdb\25af27322f8b8e51a8b6e906f8c28cf1\WdF01000.pdb

Donde vemos muchos módulos con:

```
mupupdate_GenuineIntel      (pdb symbols)
kd_02_8086                  (pdb symbols)
kdccom                      (pdb symbols)
CLFS                        (pdb symbols)
cng                         (pdb symbols)
win32k                     (pdb symbols)
CI                          (pdb symbols)
```

Eso confirma que la carpeta **C:\Symbols** está funcionando y que WinDbg está descargando/cargando símbolos correctamente.

4.5 Comprobamos el acceso a estructuras internas del kernel

```
dt nt!_DRIVER_OBJECT
```

```
kd> dt nt!_DRIVER_OBJECT
+0x000 Type           : Int2B
+0x002 Size           : Int2B
+0x008 DeviceObject   : Ptr64 _DEVICE_OBJECT
+0x010 Flags          : Uint4B
+0x018 DriverStart    : Ptr64 Void
+0x020 DriverSize     : Uint4B
+0x028 DriverSection  : Ptr64 Void
+0x030 DriverExtension : Ptr64 _DRIVER_EXTENSION
+0x038 DriverName     : _UNICODE_STRING
+0x048 HardwareDatabase : Ptr64 _UNICODE_STRING
+0x050 FastIoDispatch : Ptr64 _FAST_IO_DISPATCH
+0x058 DriverInit     : Ptr64 long
+0x060 DriverStartIo  : Ptr64 void
+0x068 DriverUnload   : Ptr64 void
+0x070 MajorFunction  : [28] Ptr64 long
```

Locals		
Name	Value	Type

Threads		
TID	Index	Thread
0xc	0x0	nt!KiSwapContext+0x76 (ffff802`ad6b1276)
0x10	0x0	nt!KiSwapContext+0x76 (ffff802`ad6b1276)
0x14	0x0	nt!KiSwapContext+0x76 (ffff802`ad6b1276)

WinDbg muestra la estructura completa:

```
+0x000 Type
+0x002 Size
+0x008 DeviceObject
+0x018 DriverStart
+0x020 DriverSize
+0x028 DriverSection
+0x038 DriverName
+0x050 FastIoDispatch
+0x068 DriverUnload
+0x070 MajorFunction
```

Eso significa que WinDbg ya puede resolver tipos internos del kernel, que es justo lo que necesitamos para analizar drivers y estructuras en Ring 0.

4.6 Probamos algunas estructuras para ver si todo funciona bien

The screenshot shows the WinDbg interface with the following content:

```
kd> dt nt!_DRIVER_OBJECT
+0x000 Type : Int2B
+0x002 Size : Int2B
+0x008 DeviceObject : Ptr64 _DEVICE_OBJECT
+0x010 Flags : Uint4B
+0x018 DriverStart : Ptr64 Void
+0x020 DriverSize : Uint4B
+0x028 DriverSection : Ptr64 Void
+0x030 DriverExtension : Ptr64 _DRIVER_EXTENSION
+0x038 DriverName : _UNICODE_STRING
+0x048 HardwareDatabase : Ptr64 _UNICODE_STRING
+0x050 FastIoDispatch : Ptr64 _FAST_IO_DISPATCH
+0x058 DriverInit : Ptr64 long
+0x060 DriverStartIo : Ptr64 void
+0x068 DriverUnload : Ptr64 void
+0x070 MajorFunction : [28] Ptr64 long

kd> dt nt!_EPROCESS
Symbol nt!_EPROCESS not found.

kd> dt nt!_UNICODE_STRING
+0x000 Length : Uint2B
+0x002 MaximumLength : Uint2B
+0x008 Buffer : Ptr64 Wchar

kd> dt nt!_LIST_ENTRY
+0x000 Flink : Ptr64 _LIST_ENTRY
+0x008 Blink : Ptr64 _LIST_ENTRY
```

Below the command window, the 'Locals' pane is empty. The 'Threads' pane shows a list of threads:

TID	Index	Thread
0xc	0x0	nt!KiSwapContext+0x76 (ffff802`ad6b1276)
0x10	0x0	nt!KiSwapContext+0x76 (ffff802`ad6b1276)
0x14	0x0	nt!KiSwapContext+0x76 (ffff802`ad6b1276)

At the bottom, the 'Threads' tab is selected in the 'Locals' pane.

Vemos un fallo con EPROCESS:

```
dt nt!_EPROCESS
```

Pero al buscar con comodín:

```
dt nt!*EPROCESS*
```

```
kd> dt nt!*EPROCESS*
ntkrnlmp!_EPROCESS_QUOTA_BLOCK
ntkrnlmp!_EPROCESS
ntkrnlmp!_EPROCESS_VALUES
fffff802adb01811 ntkrnlmp!NtAcquireProcessActivityReference$filt$0
fffff802ad8ff0d8 ntkrnlmp!PspRemoveProcessFromJobChain
fffff802ada8cf1c ntkrnlmp!PspSetCreateProcessNotifyRoutine
fffff802adba8000 ntkrnlmp!VfPendingMoreProcessingRequired
fffff802ad2f44d0 ntkrnlmp!KiUpdateProcessConcurrencyCounts
fffff802ad391f1c ntkrnlmp!HalpContinueProcessingWaitQueue
fffff802ad424ab0 ntkrnlmp!MiChangeProcessCommitment
fffff802ada48098 ntkrnlmp!KeInitializeProcess
fffff802adb02267 ntkrnlmp!NtCreateProcessStateChange$filt$0
fffff802adb0229b ntkrnlmp!NtCreateProcessStateChange$filt$1
fffff802ada3297c ntkrnlmp!EtwpWriteProcessStarted
fffff802ad418ed0 ntkrnlmp!PopDiagTraceProcessorThrottlePerfTrack
fffff802adaffcd8 ntkrnlmp!PspCreateProcess$filt$0
fffff802ada4d748 ntkrnlmp!ObCheckRefTraceProcess
fffff802ada5bb00 ntkrnlmp!EtwTiLogSuspendResumeProcess
fffff802ad9b9110 ntkrnlmp!PsQueryRuntimeProcess
fffff802ad8feb98 ntkrnlmp!PspRundownSingleProcess
fffff802ad3f3cf0 ntkrnlmp!KiPreprocessFault
fffff802ad8f23dc ntkrnlmp!EtwpWriteProcessEvent
fffff802ada7e4bc ntkrnlmp!MiChangeProcessPhysicalPages
fffff802ada72d44 ntkrnlmp!PpmEventTraceProcessorPerformanceDomainRundown
fffff802ad8f4df0 ntkrnlmp!PspTerminateProcess
```

WinDbg ha encontrado el tipo bajo el nombre:

```
ntkrnlmp!_EPROCESS
```

Existen símbolos/tipos relacionados con EPROCESS, entre ellos:

```
ntkrnlmp!_EPROCESS
ntkrnlmp!_EPROCESS_QUOTA_BLOCK
ntkrnlmp!_EPROCESS_VALUES
```

La línea importante es:

```
ntkrnlmp!_EPROCESS
```

Eso significa que la estructura sí está disponible, pero aparece asociada al módulo real cargado:

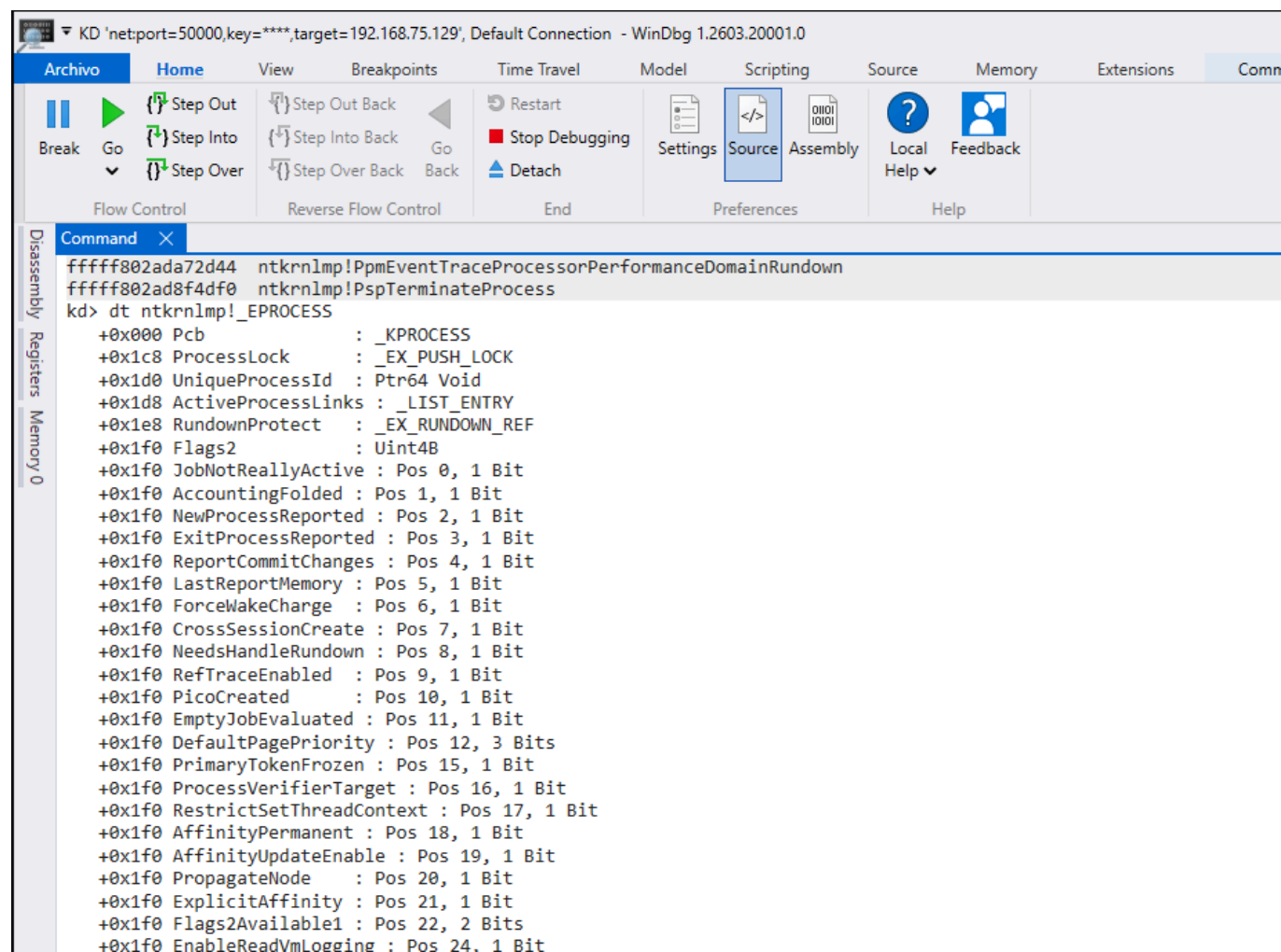
```
ntkrnlmp
```

no necesariamente como:

```
nt!_EPROCESS
```

El módulo del kernel se está resolviendo como `ntkrnlmp.exe`, que es el kernel multiprocesador de Windows. Por eso podemos usar directamente:

```
dt ntkrnlmp!_EPROCESS
```



Objetivo: explicar qué campos pueden ser relevantes para detectar manipulación.

4.7 Listamos el directorio de objetos de drivers

```
!object \Driver
```

Archivo

Home

View

Breakpoints

Time Travel

Model

Scripting

Source

Memory

Extensions

Command

Break

Go

Step Out

Step Into

Step Over

Step Out Back

Step Into Back

Step Over Back

Restart

Stop Debugging

Detach

Settings

Source

Assembly

Local Help

Feedback

Flow Control

Reverse Flow Control

End

Preferences

Help

Disassembly

Registers

Memory 0

kd> !object \Driver

Object: fffffa30d51383b30 Type: (fffffe10712694000) Directory

ObjectHeader: fffffa30d51383b00 (new version)

HandleCount: 0 PointerCount: 120

Directory Object: fffffa30d5129de10 Name: Driver

Target is not responding. Attempting to reconnect...

Hash	Address	Type	Name
00	fffffe107150e7e10	Driver	fvevol
	fffffe107150e2e10	Driver	vdvrvroot
01	fffffe10715de0d20	Driver	usbuhci
	fffffe10715b2bdc0	Driver	NetBT
	fffffe107127a7d90	Driver	acpiex
	fffffe107127a3e10	Driver	Wdf01000
02	fffffe10717bdf20	Driver	WdNisDrv
	fffffe1071767f350	Driver	mpsdrv
	fffffe10715123e10	Driver	storahci
03	fffffe10716f9b060	Driver	MMCSS
	fffffe107127b7e40	Driver	lltdio
	fffffe10712d5dcf0	Driver	RegistryHiveCacheDriver
	fffffe10712d514a0	Driver	bam
	fffffe10712d52d20	Driver	Psched
	fffffe10715b1db10	Driver	BasicRender
	fffffe10715b6eb10	Driver	disk
04	fffffe10712688720	Driver	HTTP
	fffffe10715120e10	Driver	stornvme
05	fffffe10712d51060	Driver	WscVReg
06	fffffe10716179d00	Driver	monitor
	fffffe10715de06c0	Driver	usbbehci
	fffffe10712d51280	Driver	ahcache
	fffffe10715b50bd0	Driver	vmrawdsk
	fffffe107150e0e10	Driver	iorate
	fffffe10715063e10	Driver	pcw
07	fffffe10715d7f430	Driver	Ucx01000
	fffffe10715de04a0	Driver	USBXHCI
	fffffe10715063e10	Driver	---

kd>

4.8 Probamos con el driver WdNisDrv

WdNisDrv corresponde a un driver relacionado con Microsoft Defender, concretamente con funcionalidades de inspección/protección de red.

```
!drvobj \Driver\WdNisDrv 2
```

Archivo

Home

View

Breakpoints

Time Travel

Model

Scripting

Source

Memory

Extensions

Command

Break

Go

Step Out

Step Into

Step Over

Step Out Back

Step Into Back

Step Over Back

Back

Restart

Stop Debugging

Detach

Settings

Source

Assembly

Local Help

Feedback

Flow Control

Reverse Flow Control

End

Preferences

Help

Disassembly

Registers

Memory 0

Command

36

ffffe10715de18e0

Driver

vmmouse

ffffe10715b68e20

Driver

afunix

ffffe10715de1280

Driver

vm3dmp

ffffe10715c69dd0

Driver

CompositeBus

ffffe10715a77d40

Driver

ws2ifsl

ffffe107150f6e10

Driver

EhStorClass

ffffe10712707820

Driver

ACPI

kd> !drvobj \Driver\WdNisDrv 2

Driver object (ffffe10717bdf20) is for:

Target is not responding. Attempting to reconnect...

[\Driver\WdNisDrv](#)

DriverEntry: fffff80541b6b000 WdNisDrv

DriverStartIo: 00000000

DriverUnload: fffff80541b5d560 WdNisDrv

AddDevice: 00000000

Dispatch routines:

[00]	IRP_MJ_CREATE	fffff80541b5d730	WdNisDrv+0xd730
[01]	IRP_MJ_CREATE_NAMED_PIPE	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[02]	IRP_MJ_CLOSE	fffff80541b5d6d0	WdNisDrv+0xd6d0
[03]	IRP_MJ_READ	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[04]	IRP_MJ_WRITE	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[05]	IRP_MJ_QUERY_INFORMATION	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[06]	IRP_MJ_SET_INFORMATION	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[07]	IRP_MJ_QUERY_EA	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[08]	IRP_MJ_SET_EA	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[09]	IRP_MJ_FLUSH_BUFFERS	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[0a]	IRP_MJ_QUERY_VOLUME_INFORMATION	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[0b]	IRP_MJ_SET_VOLUME_INFORMATION	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[0c]	IRP_MJ_DIRECTORY_CONTROL	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[0d]	IRP_MJ_FILE_SYSTEM_CONTROL	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[0e]	IRP_MJ_DEVICE_CONTROL	fffff80541b53fa0	WdNisDrv+0x3fa0
[0f]	IRP_MJ_INTERNAL_DEVICE_CONTROL	fffff805aafc68e0	nt!IopInvalidDeviceRequest
[10]	IRP_MJ_SHUTDOWN	fffff805aafc68e0	nt!IopInvalidDeviceRequest

4.9. Listamos los procesos activos desde la perspectiva del kernel.

```
!process 0 0
```

KD 'netport=50000,key=****,target=192.168.75.129', Default Connection - WinDbg 1.2603.20001.0

Archivo Home View Breakpoints Time Travel Model Scripting Source Memory Extension

Break Go Step Out Step Out Back Step Into Step Into Back Step Over Step Over Back Go Back Restart Stop Debugging Settings Source Assembly Local Help Feedback

Flow Control Reverse Flow Control End Preferences Help

Command X

```
ffff8702e8c75cf0 Driver CompositeBus
ffff8702e8a78d40 Driver ws2ifsl
ffff8702e80f6e10 Driver EhStorClass
ffff8702e670b060 Driver ACPI
```

kd> !process 0 0
Target is not responding. Attempting to reconnect...
**** NT ACTIVE PROCESS DUMP ****

PROCESS fffff8702e6649040
SessionId: none Cid: 0004 Peb: 00000000 ParentCid: 0000
DirBase: 001ae000 ObjectTable: fffff9705672b2e40 HandleCount: 1805.
Image: System

PROCESS fffff8702e673a080
SessionId: none Cid: 004c Peb: 00000000 ParentCid: 0004
DirBase: 102711000 ObjectTable: fffff97056729a0c0 HandleCount: 0.
Image: Registry

PROCESS fffff8702e8e49040
SessionId: none Cid: 0184 Peb: d50b5f2000 ParentCid: 0004
DirBase: 24644e000 ObjectTable: fffff970567aaafcc0 HandleCount: 58.
Image: smss.exe

PROCESS fffff8702e962d140
SessionId: none Cid: 0230 Peb: 6d834cd000 ParentCid: 0214
DirBase: 246f8c000 ObjectTable: fffff970567e59f40 HandleCount: 491.
Image: csrss.exe

PROCESS fffff8702e9a75080
SessionId: none Cid: 027c Peb: bb9a64e000 ParentCid: 0214
DirBase: 251087000 ObjectTable: fffff970567e59c00 HandleCount: 165.
Image: wininit.exe

PROCESS fffff8702e9a73140
SessionId: none Cid: 0284 Peb: 83ee3ae000 ParentCid: 0274
DirBase: 2533b4000 ObjectTable: fffff970567e583c0 HandleCount: 155.

kd>

Locals Watch Thread:

Cada bloque corresponde a un proceso. Por ejemplo:

```
PROCESS fffff8702e6649040
SessionId: none Cid: 0004 Peb: 00000000 ParentCid: 0000
DirBase: 001ae000 ObjectTable: fffff9705672b2e40 HandleCount: 1805
Image: System
```


Campo	Ejemplo en WinDbg	Descripción	Utilidad en el análisis
PROCESS	PROCESS ffff8702e6649040	Dirección del objeto EPROCESS del proceso en memoria del kernel.	Permite identificar la estructura interna que representa al proceso en el kernel y realizar un análisis más profundo sobre él.
Cid	Cid: 0004	Identificador del proceso, equivalente al PID.	Permite identificar cada proceso. Por ejemplo: 0004 corresponde a System, 0184 a smss.exe, 0230 a csrss.exe y 027c a wininit.exe.
ParentCid	ParentCid: 0004	Identificador del proceso padre.	Permite analizar relaciones entre procesos. Por ejemplo, si smss.exe tiene como ParentCid el valor 0004, significa que su proceso padre es System.
Peb	Peb: 00000000	Dirección del Process Environment Block, una estructura utilizada en modo usuario.	En procesos puramente de sistema o muy tempranos, como System o Registry, puede aparecer como 00000000. En procesos de usuario suele aparecer una dirección válida, por ejemplo d50b5f2000.
DirBase	DirBase: 24644e000	Base del directorio de páginas del proceso. Está relacionada con la traducción de memoria virtual a memoria física.	Aporta información de bajo nivel sobre el espacio de direcciones del proceso.
ObjectTable	ObjectTable: ffff970567aafcc0	Dirección de la tabla de objetos/handles del proceso.	Permite al sistema gestionar los recursos abiertos por el proceso, como ficheros, claves de registro, procesos, hilos u otros objetos.
HandleCount	HandleCount: 58	Número de handles abiertos por el proceso.	Puede ayudar a observar la cantidad de recursos que mantiene abiertos un proceso.
Image	Image: smss.exe	Nombre de la imagen o ejecutable asociado al proceso.	Permite identificar de forma legible qué proceso se está analizando.

El comando `!process 0 0` permite enumerar los procesos activos desde el contexto del kernel. La salida muestra la dirección del objeto `EPROCESS`, el identificador del proceso (`Cid`), el identificador del proceso padre (`ParentCid`), la dirección del `PEB`, la tabla de objetos y el nombre de la imagen del proceso. Esta información resulta útil para detectar posibles anomalías relacionadas con ocultación o manipulación de procesos, especialmente en escenarios donde un rootkit pueda modificar estructuras internas del kernel.

4.6 Indicadores de anomalía

Durante el análisis manual con WinDbg, no basta con obtener información sobre procesos, módulos o drivers cargados. Es necesario **interpretar los resultados y valorar si alguno de los elementos observados puede considerarse anómalo** desde el punto de vista de la seguridad. Un indicador de anomalía no implica necesariamente la presencia de malware, pero sí representa una señal que debería ser revisada con mayor profundidad.

En el contexto de este trabajo, los indicadores de anomalía se centran principalmente en drivers de kernel, rutinas de despacho, módulos cargados y estructuras internas del sistema operativo.

Uno de los primeros indicadores que debe revisarse es la presencia de un **driver cargado desde una ruta inusual**. Los drivers legítimos de Windows suelen encontrarse en rutas conocidas, como `C:\Windows\System32\drivers`. Si un driver aparece asociado a una ubicación temporal, a un directorio de usuario, a una ruta poco habitual o a una carpeta con permisos débiles, puede tratarse de un elemento sospechoso. Esta comprobación permite detectar controladores cargados desde ubicaciones que no encajan con el comportamiento esperado del sistema.

Otro indicador relevante es el **nombre sospechoso del driver o del módulo**. Algunos drivers maliciosos utilizan nombres aleatorios, cadenas sin significado, nombres que intentan imitar componentes legítimos de Windows o denominaciones muy parecidas a las de drivers reales. Por ejemplo, un nombre que se parezca visualmente a un driver del sistema pero que no coincida exactamente con él podría ser una técnica de camuflaje. En estos casos, es importante comparar el nombre del objeto de driver, el nombre del módulo y la ruta del fichero en disco.

Un punto especialmente importante es la revisión de las **dispatch routines** del `DRIVER_OBJECT`. Estas rutinas se encuentran en el campo `MajorFunction` y determinan qué funciones ejecutará el driver cuando reciba diferentes tipos de solicitudes `IRP`. En un comportamiento normal, estas direcciones deberían apuntar al propio módulo del driver o a funciones legítimas del kernel, como `nt!IopInvalidDeviceRequest` cuando una operación no está implementada. Si una rutina apunta a una dirección fuera del rango esperado, a una región de memoria desconocida o a un módulo no relacionado, podría ser un indicio de hooking o manipulación del driver.

También debe tenerse en cuenta la **ausencia de símbolos**. Que un módulo no tenga símbolos cargados no significa necesariamente que sea malicioso, ya que muchos drivers de terceros no disponen de símbolos públicos. Sin embargo, desde el punto de vista del análisis, la ausencia de símbolos dificulta la interpretación del comportamiento del módulo. Si además se combina con otros factores, como un nombre extraño, una ruta inusual o punteros anómalos, puede aumentar el nivel de sospecha.

Otro indicador importante son las **estructuras incoherentes**. Al inspeccionar estructuras internas como `DRIVER_OBJECT`, `EPROCESS`, `LIST_ENTRY` o `UNICODE_STRING`, pueden aparecer valores que no encajan con el estado esperado del sistema. Por ejemplo, punteros nulos donde debería existir una dirección válida, enlaces inconsistentes en listas, campos con valores anómalos o direcciones que no pertenecen a ningún módulo conocido. Este tipo de incoherencias puede estar relacionado con errores, corrupción de memoria o manipulación intencionada por parte de malware en modo kernel.

También es relevante detectar **módulos cargados sin información clara**. En WinDbg, comandos como `!m` o `!mv` permiten listar módulos y obtener detalles sobre su dirección base, tamaño, nombre de imagen y símbolos. Un módulo que aparece cargado pero no ofrece información suficiente, no tiene una ruta clara o no puede asociarse fácilmente a un componente legítimo del sistema debería ser analizado con más detalle. En un escenario real, este tipo de módulo podría corresponder a un driver sospechoso, a un componente oculto parcialmente o a código cargado de forma no habitual.

Estos indicadores se relacionan directamente con técnicas utilizadas por rootkits. Por ejemplo, una técnica de **hooking** puede modificar punteros de funciones para redirigir la ejecución hacia código controlado por el atacante. Esto podría observarse en una dispatch routine que apunta fuera del rango del driver esperado. De forma similar, técnicas como **DKOM** pueden alterar estructuras internas del kernel para ocultar procesos, drivers u otros objetos. En estos casos, la anomalía no se detecta únicamente por la presencia de un fichero sospechoso, sino por la incoherencia entre lo que el sistema debería mostrar y lo que realmente aparece en memoria.

En la práctica realizada, el análisis del driver `WdNisDrv` mostró un comportamiento coherente. Algunas rutinas de despacho apuntaban al propio módulo `WdNisDrv`, mientras que otras apuntaban a `nt!IopInvalidDeviceRequest`, función legítima del kernel utilizada para solicitudes no implementadas por el driver. Este resultado se considera normal. Sin embargo, la misma metodología permitiría detectar comportamientos sospechosos si alguna de esas rutinas apuntara a memoria desconocida, a un módulo no esperado o a una dirección fuera del rango del driver.

La siguiente tabla resume algunos indicadores de anomalía relevantes durante el análisis manual:

Indicador	Descripción	Posible interpretación
Driver cargado desde una ruta inusual	El controlador aparece en una ubicación distinta a las rutas habituales del sistema.	Posible driver no legítimo, cargado manualmente o utilizado por malware.
Nombre sospechoso	El módulo utiliza un nombre aleatorio, confuso o similar al de un componente legítimo.	Posible intento de camuflaje o suplantación.
Dispatch routines fuera de rango	Una rutina de MajorFunction apunta fuera del módulo esperado.	Posible hooking, manipulación del DRIVER_OBJECT o redirección de ejecución.
Ausencia de símbolos	El módulo no dispone de símbolos públicos o no pueden cargarse correctamente.	No implica malware por sí solo, pero dificulta el análisis y puede aumentar la sospecha si se combina con otros indicadores.
Estructuras incoherentes	Campos internos con valores inesperados, punteros nulos o listas alteradas.	Posible corrupción, error del driver o manipulación por malware.
Módulo sin información clara	El módulo aparece cargado, pero no se identifica correctamente su origen o propósito.	Posible componente sospechoso o cargado de forma no habitual.
Relación con técnicas de rootkit	Presencia de patrones asociados a DKOM, hooking o manipulación de estructuras.	Posible actividad maliciosa en Ring 0.

5. Caso de estudio: `mimidrv.sys`

Para aplicar los conceptos anteriores a un escenario concreto, este trabajo toma como caso de estudio `mimidrv.sys`, el driver de kernel asociado a Mimikatz. La elección de este componente resulta adecuada porque permite analizar un ejemplo real de driver con capacidades sensibles en `Ring 0`, sin que sea necesario ejecutar malware real ni comprometer el sistema de laboratorio.

`mimidrv.sys` se estudia en este trabajo desde una perspectiva defensiva y académica. El objetivo no es utilizarlo con fines ofensivos, sino comprender por qué un driver de kernel puede representar un riesgo relevante para la seguridad de Windows y qué elementos podrían ser inspeccionados mediante WinDbg para detectar comportamientos anómalos.

5.1 Qué es Mimikatz

Mimikatz es una herramienta ampliamente conocida en el ámbito de la ciberseguridad, especialmente por su capacidad para interactuar con mecanismos de autenticación de Windows y extraer información sensible relacionada con credenciales. Aunque puede utilizarse en entornos controlados para auditorías, formación y análisis defensivo, también ha sido utilizada de forma maliciosa en escenarios de post-explotación.

Uno de los usos más conocidos de Mimikatz es la obtención de contraseñas, hashes, tickets Kerberos y otro material relacionado con credenciales almacenadas o gestionadas por Windows. Para ello, históricamente ha tenido especial interés en el proceso `lsass.exe`, que corresponde al servicio `Local Security Authority Subsystem Service`. Este proceso es crítico para el sistema operativo, ya que participa en tareas de autenticación, validación de usuarios, gestión de sesiones y aplicación de políticas de seguridad.

Debido a la sensibilidad de la información gestionada por `lsass.exe`, las versiones modernas de Windows incorporan diferentes mecanismos de protección para dificultar el acceso directo a su memoria. Esto limita la capacidad de herramientas en modo usuario para leer información sensible de este proceso. En este contexto aparece la importancia de componentes en modo kernel, como `mimidrv.sys`.

Desde una perspectiva de análisis, Mimikatz permite comprender la diferencia entre las capacidades de una herramienta en modo usuario y las capacidades adicionales que puede obtener cuando se apoya en un driver de kernel. Esta diferencia es fundamental para entender por qué la detección de drivers sospechosos, la revisión de objetos `DRIVER_OBJECT` y el análisis de estructuras internas del kernel son tareas relevantes en la detección de anomalías a bajo nivel.

5.2 Qué es mimidrv.sys

mimidrv.sys es el driver de modo kernel asociado a Mimikatz. A diferencia del ejecutable principal de la herramienta, que opera en modo usuario, este componente se carga como un controlador del sistema y se ejecuta en **Ring 0**. Esto le permite interactuar con partes internas de Windows que no están disponibles para una aplicación convencional ejecutada en **Ring 3**.

La extensión **.sys** indica que se trata de un driver de Windows. Los drivers de este tipo se cargan en el espacio del kernel y pueden acceder a recursos privilegiados del sistema operativo, como memoria del kernel, estructuras internas, objetos de procesos, mecanismos de seguridad y rutinas de bajo nivel.

En el caso de **mimidrv.sys**, su interés principal se encuentra en que proporciona a Mimikatz capacidades que no serían posibles únicamente desde **modo usuario**. Windows implementa diferentes mecanismos de protección para impedir que procesos normales accedan libremente a componentes críticos del sistema, como **lsass.exe** o determinadas estructuras protegidas. Al operar en modo kernel, un driver puede situarse en una posición mucho más privilegiada y realizar acciones que superan las restricciones habituales del modo usuario.

Desde una perspectiva defensiva, **mimidrv.sys** resulta relevante porque representa un ejemplo de driver con capacidades sensibles. Su análisis permite comprender cómo un componente en Ring 0 puede interactuar con mecanismos internos del sistema operativo y por qué la presencia de drivers no esperados, maliciosos o abusados puede suponer un riesgo elevado.

En un análisis con WinDbg, un driver como **mimidrv.sys** puede estudiarse observando cómo aparece cargado en memoria, qué objeto **DRIVER_OBJECT** tiene asociado, cuáles son sus rutinas de despacho, qué rango de memoria ocupa y qué punteros o estructuras están relacionados con él. Estos elementos permiten al analista evaluar si el driver presenta características esperadas o si existen indicios de comportamiento anómalo.

Uno de los aspectos más interesantes sería revisar las rutinas almacenadas en el campo **MajorFunction** del **DRIVER_OBJECT**. Estas rutinas determinan cómo responde el driver a distintas solicitudes del sistema. Si alguna de estas funciones apunta a una región de memoria inesperada, a un módulo no relacionado o a una zona no identificada, podría considerarse un indicador de posible manipulación o comportamiento sospechoso.

También resulta relevante comprobar la dirección de inicio del driver mediante campos como **DriverStart** y su tamaño mediante **DriverSize**. Estos valores permiten establecer el rango de memoria ocupado por el controlador. A partir de ahí, se pueden comparar otras direcciones asociadas al driver para verificar si se encuentran dentro de un rango coherente.

5.3 Qué relación tiene con LSASS/PPL

La relación entre `mimidrv.sys`, `LSASS` y `PPL` se encuentra en el nivel de privilegios necesario para interactuar con determinados procesos protegidos de Windows. `LSASS`, cuyo nombre completo es *Local Security Authority Subsystem Service*, es un proceso crítico del sistema operativo encargado de funciones relacionadas con la autenticación, la aplicación de políticas de seguridad y la gestión de sesiones de usuario. Por su importancia, es uno de los procesos más sensibles desde el punto de vista de la seguridad.

Históricamente, herramientas como Mimikatz han estado muy asociadas al análisis de `LSASS` porque este proceso puede contener material sensible relacionado con la autenticación. Sin embargo, en versiones modernas de Windows, Microsoft ha incorporado mecanismos de protección para dificultar que procesos convencionales en modo usuario puedan acceder libremente a `LSASS`. Uno de estos mecanismos es `PPL`, siglas de *Protected Process Light*.

`PPL` es una tecnología de protección que permite ejecutar determinados procesos con un nivel adicional de restricción. Cuando `LSASS` se ejecuta como proceso protegido, otros procesos de menor nivel de confianza no pueden interactuar con él de forma normal, aunque el usuario tenga privilegios administrativos. Esto limita acciones como abrir manejadores con permisos elevados, leer memoria del proceso o manipularlo desde herramientas convencionales en modo usuario.

Esta protección es especialmente relevante porque marca una diferencia clara entre lo que puede hacer un proceso en `Ring 3` y lo que puede hacer un componente en `Ring 0`. Un ejecutable normal, como `mimikatz.exe`, se ejecuta en modo usuario y está sujeto a las comprobaciones de seguridad impuestas por el sistema operativo. En cambio, un driver de kernel como `mimidrv.sys` se ejecuta en `Ring 0`, en el mismo nivel de privilegio que el kernel de Windows y otros controladores del sistema.

5.4 Ejecución controlada de Mimikatz/mimidrv.sys y análisis desde WinDbg

Ejecutaremos Mimikatz en la máquina destino para observar su proceso y, si el entorno lo permite, detectar el driver mimidrv.sys desde WinDbg en la máquina analista.

Antes de ejecutar Mimikatz se creó un snapshot de ambas máquinas virtuales. La prueba se realizó en un entorno aislado, sin datos reales ni credenciales sensibles, con finalidad exclusivamente académica y defensiva.

Observaremos:

- El proceso mimikatz.exe.
- La posible carga del driver mimidrv.sys.
- Los cambios visibles desde WinDbg.

A) Tomamos una línea base antes de ejecutar Mimikatz

Antes de ejecutar Mimikatz tomamos una línea base del sistema:

```
vertarget
lmv m nt
!process 0 0
!object \Driver
lm
```


B) Deshabilitamos antivirus en maquina destino

Deshabilitamos el antivirus en Windows 11 y añadimos las siguientes exclusiones:

Exclusiones

Agregar o quitar los elementos que quieras excluir de los análisis de Antivirus de Microsoft Defender.

¿Tienes alguna pregunta?

[Obtener ayuda](#)

+ Agregar exclusión

Ayuda a mejorar el servicio
Seguridad de Windows

[Envíanos tus comentarios](#)

C:\Users\usuario2\Desktop
Carpeta

Cambiar la configuración de
privacidad

.exe
Tipo de archivo

Visualiza y cambia la configuración
de privacidad del dispositivo
Windows 11 Home.

[Configuración de privacidad](#)

[Panel de privacidad](#)

[Declaración de privacidad](#)

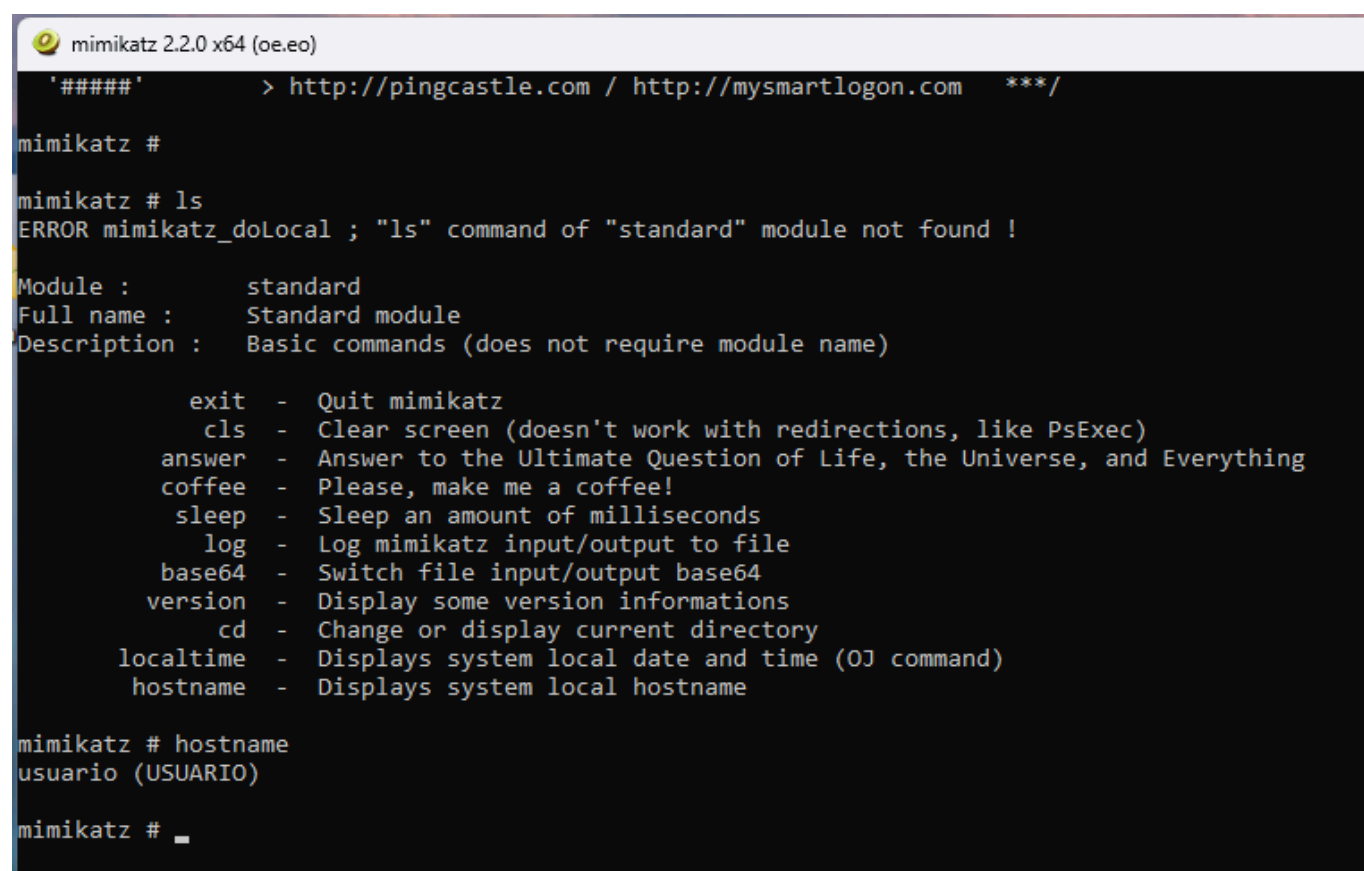
C) Copiamos Mimikatz a la máquina destino

Del repositorio [Github](#) descargamos:

```
mimikatz.exe  
mimidrv.sys
```

D) Ejecutamos Minikatz

Condición	Explicación
El fichero <code>mimidrv.sys</code> debe existir	Normalmente debe estar junto a <code>mimikatz.exe</code> o en una ruta accesible.
Mimikatz debe ejecutarse como administrador	Cargar un driver requiere privilegios elevados.
El driver debe cargarse explícitamente	No se carga solo al abrir Mimikatz.
Windows debe permitir la carga	Windows 11 actualizado puede bloquearlo por firma, HVCI, Defender, políticas de seguridad, etc.



```

mimikatz 2.2.0 x64 (oe.eo)
'#####' > http://pingcastle.com / http://mysmartlogon.com ***/

mimikatz #
mimikatz # ls
ERROR mimikatz_doLocal ; "ls" command of "standard" module not found !

Module :      standard
Full name :    Standard module
Description :  Basic commands (does not require module name)

    exit - Quit mimikatz
    cls  - Clear screen (doesn't work with redirections, like PsExec)
    answer - Answer to the Ultimate Question of Life, the Universe, and Everything
    coffee - Please, make me a coffee!
    sleep - Sleep an amount of milliseconds
    log   - Log mimikatz input/output to file
    base64 - Switch file input/output base64
    version - Display some version informations
    cd    - Change or display current directory
    localtime - Displays system local date and time (OJ command)
    hostname - Displays system local hostname

mimikatz # hostname
usuario (USUARIO)

mimikatz # _

```

Una vez ejecutado la aplicación, dentro de Mimikatz comprobamos privilegios:

```
privilege::debug
```

Vemos:

```
Privilege '20' OK
```

Cargamos mimidrv.sys: En el prompt de Mimikatz ejecutamos:

```
!+
```

```
mimikatz # hostname
usuario (USUARIO)

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # !+
[*] 'mimidrv' service not present
[+] 'mimidrv' service successfully registered
[+] 'mimidrv' service ACL to everyone
[+] 'mimidrv' service started

mimikatz #
```

Verificamos que el servicio está activo en la máquina destino:

```
sc query type= driver state= all | findstr /i mimi
sc query mimidrv
```

```
C:\Windows\System32>sc query type= driver state= all | findstr /i mimi
NOMBRE_SERVICIO: mimidrv
NOMBRE_MOSTRAR : mimikatz driver (mimidrv)

C:\Windows\System32>sc query mimidrv

NOMBRE_SERVICIO: mimidrv
        TIPO                : 1  KERNEL_DRIVER
        ESTADO                : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        CÓD_SALIDA_WIN32      : 0  (0x0)
        CÓD_SALIDA_SERVICIO: 0  (0x0)
        PUNTO_COMPROB.       : 0x0
        INDICACIÓN_INICIO    : 0x0

C:\Windows\System32>
```

E) Ejecutamos WinDbg en la maquina analista**

```
lm m mimi*
```

```
kd> lm m mimi*
Browse full module list
start          end                module name
Target is not responding. Attempting to reconnect...
kd> lm m mimi*
Browse full module list
start          end                module name
kd> lmD
start          end                module name
fffff806`74200000 fffff806`75650000 nt (pdb symbols) c:\symbols\ntkrnlmp.pdb\11D7FE79CC245612055503EE86BB0E3E1\ntkrnlmp.pdb

Unloaded modules:
fffff806`09800000 fffff806`09822000 ExecutionContext.sys
fffff806`0b330000 fffff806`0b351000 WdNisDrv.sys
fffff806`0af10000 fffff806`0af30000 NetworkPrivacyPolicy.sys
fffff806`09730000 fffff806`0974a000 dump_storport.sys
fffff806`08a00000 fffff806`08a51000 dump_storvme.sys
fffff806`08a90000 fffff806`08ab7000 dump_dumpfve.sys
fffff806`09150000 fffff806`09173000 dam.sys
fffff806`09180000 fffff806`09194000 KMPDC.sys
fffff806`08bc0000 fffff806`08bd4000 uimap.sys
fffff806`06cb0000 fffff806`06cbc000 WdBoot.sys
fffff806`08700000 fffff806`08714000 hwpolicy.sys

kd>
```

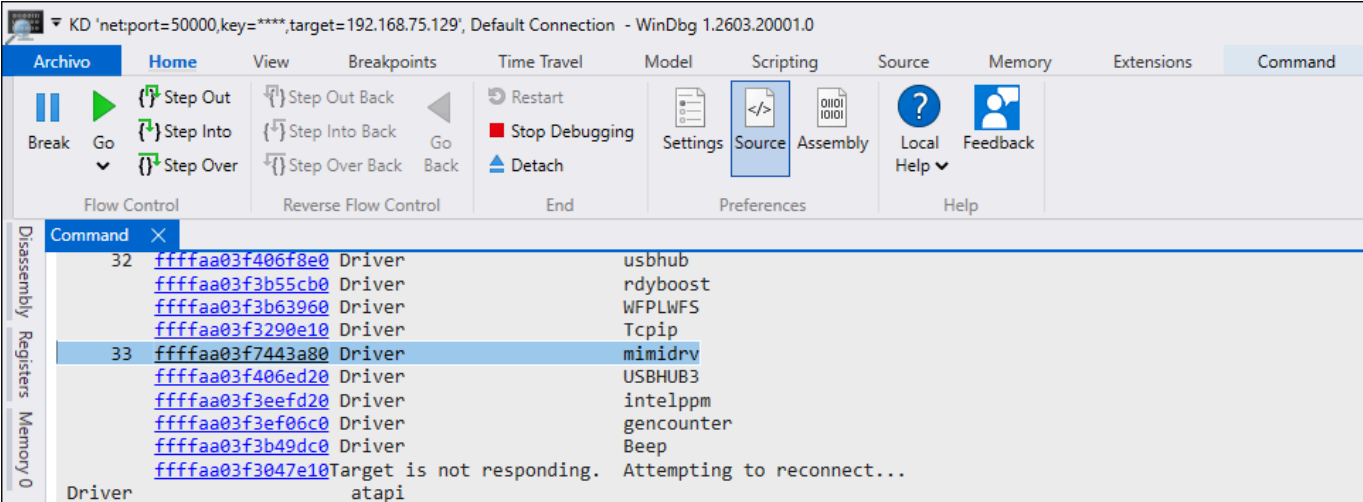
`lm m mimi*` busca módulos cargados cuyo nombre coincida con `mimi*`. Pero comprobamos que no aparece. Esto no significa automáticamente que no exista el objeto de driver.

En la máquina destino se comprueba mediante `sc query mimidrv` que el servicio de tipo `KERNEL_DRIVER` se encuentra en estado `RUNNING`. Sin embargo, desde WinDbg, el comando `lm m mimi*` no muestra ningún módulo coincidente. Por este motivo, ampliamos la comprobación revisando el espacio de nombres de objetos del kernel mediante `!object \Driver`.

F) Confirmar objeto de driver en WinDbg

Ejecutamos:

```
!object \Driver
```



En la captura aparece el objeto:

```
Driver      mimidrv
```

dentro del directorio:

```
\Driver
```

Eso confirma que **mimidrv** no sólo existe como servicio en la máquina destino, sino que también está registrado como objeto de driver en el namespace del kernel.

Evidencia	Comando	Resultado
Servicio de driver activo	sc query mimidrv	STATE: 4 RUNNING
Objeto de driver visible en kernel	!object \Driver	Aparece mimidrv
Análisis posible con WinDbg	siguiente paso	!drvobj \Driver\mimidrv 2

G) Ver las Dispatch routines

```

kd> !drvobj \Driver\mimidrv 2
Driver object (ffffaa03f7443a80) is for:
Target is not responding. Attempting to reconnect...
\Driver\mimidrv

DriverEntry: fffff8060b36b064
DriverStartIo: 00000000
DriverUnload: fffff8060b36122c
AddDevice: 00000000

Dispatch routines:
[00] IRP_MJ_CREATE fffff8060b361220
+0xfffff8060b361220
[01] IRP_MJ_CREATE_NAMED_PIPE fffff8060b361220
+0xfffff8060b361220
[02] IRP_MJ_CLOSE fffff8060b361220
+0xfffff8060b361220
[03] IRP_MJ_READ fffff8060b361220
+0xfffff8060b361220
[04] IRP_MJ_WRITE fffff8060b361220
+0xfffff8060b361220
[05] IRP_MJ_QUERY_INFORMATION fffff8060b361220
+0xfffff8060b361220
[06] IRP_MJ_SET_INFORMATION fffff8060b361220
+0xfffff8060b361220
[07] IRP_MJ_QUERY_EA fffff8060b361220
+0xfffff8060b361220
[08] IRP_MJ_SET_EA fffff8060b361220
+0xfffff8060b361220
[09] IRP_MJ_FLUSH_BUFFERS fffff8060b361220
+0xfffff8060b361220
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION fffff8060b361220
+0xfffff8060b361220
[0b] IRP_MJ_SET_VOLUME_INFORMATION fffff8060b361220
+0xfffff8060b361220
[0c] IRP_MJ_DIRECTORY_CONTROL fffff8060b361220
+0xfffff8060b361220
[0d] IRP_MJ_FILE_SYSTEM_CONTROL fffff8060b361220
+0xfffff8060b361220
[0e] IRP_MJ_DEVICE_CONTROL fffff8060b361434
+0xfffff8060b361434
[0f] IRP_MJ_INTERNAL_DEVICE_CONTROL fffff8060b361220
+0xfffff8060b361220
[10] IRP_MJ_SHUTDOWN fffff8060b361220
+0xfffff8060b361220
[11] IRP_MJ_LOCK_CONTROL fffff8060b361220
+0xfffff8060b361220
[12] IRP_MJ_CLEANUP fffff8060b361220
+0xfffff8060b361220
[13] IRP_MJ_CREATE_MAILSLLOT fffff8060b361220

```

```

+0xffffffff8060b361220
[14] IRP_MJ_QUERY_SECURITY          fffff8060b361220
+0xffffffff8060b361220
[15] IRP_MJ_SET_SECURITY            fffff8060b361220
+0xffffffff8060b361220
[16] IRP_MJ_POWER                   fffff8060b361220
+0xffffffff8060b361220
[17] IRP_MJ_SYSTEM_CONTROL          fffff8060b361220
+0xffffffff8060b361220
[18] IRP_MJ_DEVICE_CHANGE            fffff8060b361220
+0xffffffff8060b361220
[19] IRP_MJ_QUERY_QUOTA              fffff8060b361220
+0xffffffff8060b361220
[1a] IRP_MJ_SET_QUOTA               fffff8060b361220
+0xffffffff8060b361220
[1b] IRP_MJ_PNP                     fffff806745c68e0
nt!IopInvalidDeviceRequest

```

Hemos conseguido inspeccionar el **DRIVER_OBJECT** de **mimidrv** desde la máquina analista con WinDbg. La evidencia principal es:

```

Driver object (ffffaa03f7443a80) is for:
\Driver\mimidrv

```

Eso confirma que **mimidrv.sys** está registrado como objeto de driver en el kernel de la máquina destino.

1. Objeto del driver:

```

Driver object (ffffaa03f7443a80) is for:
\Driver\mimidrv

```

WinDbg ha localizado el objeto **DRIVER_OBJECT** asociado al driver mimidrv. La dirección:

```
ffffaa03f7443a80
```

Esa es la dirección del objeto en memoria del kernel. Esto es importante porque permite analizar el driver desde **Ring 0** y consultar sus rutinas internas.

2. Rutinas principales del driver:

```

DriverEntry:  fffff8060b36b064
DriverStartIo: 00000000
DriverUnload:  fffff8060b36122c
AddDevice:     00000000

```

Campo	Valor	Interpretación
DriverEntry	ffffff8060b36b064	Punto de entrada del driver. Es la rutina que se ejecuta cuando el driver se carga.
DriverStartIo	00000000	No implementa rutina StartIo . No es necesariamente anómalo.
DriverUnload	ffffff8060b36122c	Rutina que se ejecuta al descargar el driver.
AddDevice	00000000	No implementa AddDevice , algo habitual en drivers que no siguen un modelo PnP clásico.

3. Dispatch routines:

La parte más importante es esta:

```
Dispatch routines:
[00] IRP_MJ_CREATE          fffffff8060b361220
[02] IRP_MJ_CLOSE          fffffff8060b361220
[03] IRP_MJ_READ           fffffff8060b361220
[04] IRP_MJ_WRITE          fffffff8060b361220
[0e] IRP_MJ_DEVICE_CONTROL fffffff8060b361434
[1b] IRP_MJ_PNP            fffffff806745c68e0
nt!IopInvalidDeviceRequest
```

La mayoría de rutinas apuntan a la misma dirección:

```
ffffff8060b361220
```

Esto sugiere que el driver usa una rutina común para muchas operaciones **IRP**. La más interesante es:

```
[0e] IRP_MJ_DEVICE_CONTROL fffffff8060b361434
```

IRP_MJ_DEVICE_CONTROL es especialmente relevante porque suele ser la rutina que gestiona peticiones **IOCTL**, es decir, comunicaciones desde modo usuario hacia el driver de kernel.

En un driver como **mimidrv**, esta rutina es importante porque puede actuar como punto de entrada para que el ejecutable en modo usuario se comunique con el componente en **Ring 0**.

Tabla resumen:

Elemento	Resultado observado	Interpretación
Objeto del driver	<code>\Driver\mimidrv</code>	El driver está registrado en el namespace del kernel.
Dirección del <code>DRIVER_OBJECT</code>	<code>fffffaa03f7443a80</code>	Dirección del objeto de driver en memoria kernel.
<code>DriverEntry</code>	<code>ffffff8060b36b064</code>	Punto de entrada del driver al cargarse.
<code>DriverUnload</code>	<code>ffffff8060b36122c</code>	Rutina de descarga del driver.
<code>DriverStartIo</code>	<code>00000000</code>	No implementa rutina <code>StartIo</code> .
<code>AddDevice</code>	<code>00000000</code>	No implementa rutina <code>AddDevice</code> .
Rutina común IRP	<code>ffffff8060b361220</code>	Usada por la mayoría de operaciones IRP.
<code>IRP_MJ_DEVICE_CONTROL</code>	<code>ffffff8060b361434</code>	Rutina especialmente relevante para comunicación mediante IOCTL.
<code>IRP_MJ_PNP</code>	<code>nt!IopInvalidDeviceRequest</code>	Petición no implementada, gestionada por rutina estándar del kernel.

H) Confirmar rango de memoria del driver

Ahora vamos a obtener el rango de memoria real del driver para comprobar si esas direcciones pertenecen al propio `mimidrv.sys`.

```
nt!_DRIVER_OBJECT fffffaa03f7443a80 DriverStart DriverSize DriverName
DriverUnload MajorFunction
```

Con eso vamos a ver:

```
DriverStart
DriverSize
DriverName
DriverUnload
MajorFunction
```

Obtenemos:

```
kd> dt nt!_DRIVER_OBJECT fffffa03f7443a80 DriverStart DriverSize DriverName
DriverUnload MajorFunction
+0x018 DriverStart   : 0xffffffff806`0b360000 Void
+0x020 DriverSize    : 0xe000
+0x038 DriverName    : _UNICODE_STRING "\Driver\mimidrv"
+0x068 DriverUnload  : 0xffffffff806`0b36122c      void +fffff8060b36122c
+0x070 MajorFunction : [28] 0xffffffff806`0b361220      long
+fffff8060b361220
```

A continuación, consultamos campos concretos del `DRIVER_OBJECT` asociado a `mimidrv` mediante el comando:

```
dt nt!_DRIVER_OBJECT fffffa03f7443a80 DriverStart DriverSize DriverName
DriverUnload MajorFunction
```

La salida muestra que el driver comenzaba en la dirección `0xffffffff8060b360000` y tiene un tamaño de `0xe000` bytes. A partir de estos valores se obtiene el rango de memoria ocupado por el driver:

```
0xffffffff8060b360000 - 0xffffffff8060b36e000
```

Este rango permite comprobar si las rutinas asociadas al driver apuntan a direcciones coherentes. En este caso, tanto `DriverUnload` (`0xffffffff8060b36122c`) como la primera entrada de `MajorFunction` (`0xffffffff8060b361220`) se encuentran dentro del rango ocupado por `mimidrv.sys`.

Además, en la salida anterior de `!drvobj \Driver\mimidrv 2` se observa que la rutina `IRP_MJ_DEVICE_CONTROL` apunta a `0xffffffff8060b361434`, dirección que también se encuentra dentro del rango del driver. Esta rutina resulta especialmente relevante porque suele gestionar las peticiones `IOCTL` enviadas desde modo usuario hacia el driver de kernel.

Desde el punto de vista de la detección, este resultado es coherente: las rutinas principales del `DRIVER_OBJECT` apuntan al propio módulo `mimidrv.sys`. Si alguna de estas direcciones hubiera apuntado fuera del rango del driver, a memoria desconocida o a un módulo no esperado, podría considerarse un indicador de posible `hooking` o manipulación del objeto de driver.

I) Confirmar rango de memoria del driver

Vamos a listar todas las entradas de **MajorFunction** directamente desde memoria:

```
kd> dps fffffaa03f7443a80+70 L1c
fffffaa03`f7443af0  ffffff806`0b361220
fffffaa03`f7443af8  ffffff806`0b361220
fffffaa03`f7443b00  ffffff806`0b361220
fffffaa03`f7443b08  ffffff806`0b361220
fffffaa03`f7443b10  ffffff806`0b361220
fffffaa03`f7443b18  ffffff806`0b361220
fffffaa03`f7443b20  ffffff806`0b361220
fffffaa03`f7443b28  ffffff806`0b361220
fffffaa03`f7443b30  ffffff806`0b361220
fffffaa03`f7443b38  ffffff806`0b361220
fffffaa03`f7443b40  ffffff806`0b361220
fffffaa03`f7443b48  ffffff806`0b361220
fffffaa03`f7443b50  ffffff806`0b361220
fffffaa03`f7443b58  ffffff806`0b361220
fffffaa03`f7443b60  ffffff806`0b361434
fffffaa03`f7443b68  ffffff806`0b361220
fffffaa03`f7443b70  ffffff806`0b361220
fffffaa03`f7443b78  ffffff806`0b361220
fffffaa03`f7443b80  ffffff806`0b361220
fffffaa03`f7443b88  ffffff806`0b361220
fffffaa03`f7443b90  ffffff806`0b361220
fffffaa03`f7443b98  ffffff806`0b361220
fffffaa03`f7443ba0  ffffff806`0b361220
fffffaa03`f7443ba8  ffffff806`0b361220
fffffaa03`f7443bb0  ffffff806`0b361220
fffffaa03`f7443bb8  ffffff806`0b361220
fffffaa03`f7443bc0  ffffff806`0b361220
fffffaa03`f7443bc8  ffffff806`745c68e0 nt!IopInvalidDeviceRequest
```

Comprobamos que la tabla **MajorFunction** del **DRIVER_OBJECT** de **mimidrv** está localizada correctamente y sus punteros son coherentes.

Parte	Significado
dps	Muestra punteros y trata de resolverlos como símbolos.
fffffaa03f7443a80	Dirección del DRIVER_OBJECT de mimidrv .
+70	Offset del campo MajorFunction dentro de _DRIVER_OBJECT .
L1c	Muestra 0x1c entradas, es decir, 28 punteros.

Para comprobar de forma directa la tabla **MajorFunction** del **DRIVER_OBJECT**, utilizamos el comando **dps** sobre la dirección base del objeto de driver más el offset **0x70**, correspondiente al campo **MajorFunction**:

```
dps fffffaa03f7443a80+70 L1c
```

La salida muestra las **28** entradas de la tabla de rutinas de despacho del driver. La mayoría de entradas apuntan a la dirección **0xffffffff8060b361220**, ubicada dentro del rango de memoria de **mimidrv.sys** (**0xffffffff8060b360000** - **0xffffffff8060b36e000**). Esto indica que dichas rutinas pertenecen al propio driver.

La entrada correspondiente a **IRP_MJ_DEVICE_CONTROL** apunta a **0xffffffff8060b361434**, también dentro del rango del driver. Esta rutina resulta especialmente relevante porque normalmente gestiona las peticiones **IOCTL** enviadas desde modo usuario hacia el driver en modo kernel.

La última entrada apunta a **nt!IopInvalidDeviceRequest**, función legítima del kernel utilizada cuando un driver no implementa una determinada operación **IRP**. Por tanto, no se observan punteros hacia regiones desconocidas ni hacia módulos inesperados.

Desde el punto de vista de detección, el resultado se considera coherente. Si alguna entrada de **MajorFunction** hubiera apuntado fuera del rango del driver o a memoria no asociada a un módulo legítimo, podría haberse considerado un posible indicador de hooking o manipulación del **DRIVER_OBJECT**.

5.7 Tabla de resultados manuales en WinDbg

Comprobación	Comando	Resultado	Interpretación
Proceso Mimikatz	!process 0 0	Aparece/no aparece mimikatz.exe	Permite observar el proceso desde kernel.
Servicio del driver	sc query mimidrv	RUNNING	El driver está iniciado en la máquina destino.
Objeto de driver	!object \Driver	Aparece mimidrv	El driver está registrado en el namespace del kernel.
DRIVER_OBJECT	!drvobj \Driver\mimidrv 2	Muestra rutinas IRP	Permite analizar dispatch routines.
Rango del driver	dt nt!_DRIVER_OBJECT ...	DriverStart y DriverSize	Permite calcular el rango válido del driver.
Tabla MajorFunction	dps DRIVER_OBJECT+0x70 L1c	Punteros dentro del rango	No se observan punteros fuera de rango.

6. Automatización mediante scripting

Vamos a realizar un script JavaScript para WinDbg que automatiza la revisión de la tabla **MajorFunction** del **DRIVER_OBJECT** de **\Driver\mimidrv**.

6.1. Objetivo del script

- Localizar el **DRIVER_OBJECT** de **\Driver\mimidrv**.
 - Obtener **DriverStart**, **DriverSize**, **DriverUnload** y **MajorFunction**.
 - Calcular el rango de memoria ocupado por **mimidrv.sys**.
 - Recorrer las 28 entradas de **MajorFunction**.
 - Marcar como coherentes las rutinas dentro del rango del driver.
 - Marcar como coherente **nt!IopInvalidDeviceRequest**.
 - Marcar como sospechoso cualquier puntero fuera del rango esperado.
-

6.2 El script

```
"use strict";

/*
    CheckMimidrvMajorFunction.js

    Script para WinDbg que automatiza la revisión de la tabla MajorFunction
    del DRIVER_OBJECT asociado a \Driver\mimidrv.

    Objetivo general:
    - Localizar el objeto DRIVER_OBJECT de \Driver\mimidrv.
    - Obtener la dirección base del driver mediante DriverStart.
    - Obtener el tamaño del driver mediante DriverSize.
    - Calcular el rango de memoria ocupado por mimidrv.sys.
    - Recorrer las 28 entradas de la tabla MajorFunction.
    - Comprobar si cada rutina apunta dentro del rango del propio driver.
    - Considerar como normal nt!IopInvalidDeviceRequest para IRPs no
    implementadas.
    - Marcar como sospechoso cualquier puntero fuera del rango esperado.

    Uso en WinDbg:
        .load jsprovider
        .scriptload C:\Users\usuario2\Desktop\CheckMimidrvMajorFunction.js
        !checkmimidrv
*/
...
...
```

Script Completo: [CheckMimidrvMajorFunction.js](#).

6.3. Funcionamiento del script

Con la máquina destino detenida en el `prompt kd>`, ejecutamos:

```
.scriptload C:\Lab\CheckMimidrvMajorFunction.js
```

A continuación lanzamos el script:

```
kd> !checkmimidrv
=====
Check MajorFunction for \Driver\mimidrv
=====
[+] DRIVER_OBJECT: fffffaa03f7443a80
[+] DriverStart : 0xfffff806`0b360000
[+] DriverSize : 0xe000
[+] DriverEnd : fffff806`0b36e000
[+] DriverUnload: 0xfffff806`0b36122c

[*] MajorFunction table:

[00] IRP_MJ_CREATE                fffff806`0b361220 OK      inside
mimidrv.sys
[01] IRP_MJ_CREATE_NAMED_PIPE    fffff806`0b361220 OK      inside
mimidrv.sys
[02] IRP_MJ_CLOSE                fffff806`0b361220 OK      inside
mimidrv.sys
[03] IRP_MJ_READ                 fffff806`0b361220 OK      inside
mimidrv.sys
[04] IRP_MJ_WRITE                fffff806`0b361220 OK      inside
mimidrv.sys
[05] IRP_MJ_QUERY_INFORMATION    fffff806`0b361220 OK      inside
mimidrv.sys
[06] IRP_MJ_SET_INFORMATION      fffff806`0b361220 OK      inside
mimidrv.sys
[07] IRP_MJ_QUERY_EA            fffff806`0b361220 OK      inside
mimidrv.sys
[08] IRP_MJ_SET_EA              fffff806`0b361220 OK      inside
mimidrv.sys
[09] IRP_MJ_FLUSH_BUFFERS       fffff806`0b361220 OK      inside
mimidrv.sys
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION fffff806`0b361220 OK      inside
mimidrv.sys
[0b] IRP_MJ_SET_VOLUME_INFORMATION fffff806`0b361220 OK      inside
mimidrv.sys
[0c] IRP_MJ_DIRECTORY_CONTROL   fffff806`0b361220 OK      inside
mimidrv.sys
[0d] IRP_MJ_FILE_SYSTEM_CONTROL fffff806`0b361220 OK      inside
mimidrv.sys
```

```

[0e] IRP_MJ_DEVICE_CONTROL          ffffff806`0b361434  OK      inside
mimidrv.sys
[0f] IRP_MJ_INTERNAL_DEVICE_CONTROL ffffff806`0b361220  OK      inside
mimidrv.sys
[10] IRP_MJ_SHUTDOWN                 ffffff806`0b361220  OK      inside
mimidrv.sys
[11] IRP_MJ_LOCK_CONTROL              ffffff806`0b361220  OK      inside
mimidrv.sys
[12] IRP_MJ_CLEANUP                  ffffff806`0b361220  OK      inside
mimidrv.sys
[13] IRP_MJ_CREATE_MAILSLLOT         ffffff806`0b361220  OK      inside
mimidrv.sys
[14] IRP_MJ_QUERY_SECURITY            ffffff806`0b361220  OK      inside
mimidrv.sys
[15] IRP_MJ_SET_SECURITY              ffffff806`0b361220  OK      inside
mimidrv.sys
[16] IRP_MJ_POWER                     ffffff806`0b361220  OK      inside
mimidrv.sys
[17] IRP_MJ_SYSTEM_CONTROL            ffffff806`0b361220  OK      inside
mimidrv.sys
[18] IRP_MJ_DEVICE_CHANGE             ffffff806`0b361220  OK      inside
mimidrv.sys
[19] IRP_MJ_QUERY_QUOTA               ffffff806`0b361220  OK      inside
mimidrv.sys
[1a] IRP_MJ_SET_QUOTA                ffffff806`0b361220  OK      inside
mimidrv.sys
[1b] IRP_MJ_PNP                      ffffff806`745c68e0  OK
standard kernel routine for unsupported IRP (nt!IopInvalidDeviceRequest)

```

```

=====
[+] Result: no suspicious MajorFunction pointers found.
=====
@$checkmimidrv()

```

KD 'netport=50000,key=****,target=192.168.75.129', Default Connection - WinDbg 1.2603.20001.0

Archivo Home View Breakpoints Time Travel Model Scripting Source Memory Extensions Command

Break Go Step Out Step Into Step Over Step Out Back Step Into Back Step Over Back Restart Stop Debugging Detach Settings Source Assembly Local Help Feedback

Flow Control Reverse Flow Control End Preferences Help

Command X

```
kd> .scriptload C:\Users\usuario2\Desktop\CheckMimidrvMajorFunction.js
JavaScript script successfully loaded from 'C:\Users\usuario2\Desktop\CheckMimidrvMajorFunction.js'
kd> !checkmimidrv

=====
Check MajorFunction for \Driver\mimidrv
=====
[+] DRIVER_OBJECT: fffffaa03f7443a80
[+] DriverStart : 0xfffff806`0b360000
[+] DriverSize : 0xe000
[+] DriverEnd : fffff806`0b36e000
[+] DriverUnload: 0xfffff806`0b36122c

[*] MajorFunction table:

[00] IRP_MJ_CREATE fffff806`0b361220 OK inside mimidrv.sys
[01] IRP_MJ_CREATE_NAMED_PIPE fffff806`0b361220 OK inside mimidrv.sys
[02] IRP_MJ_CLOSE fffff806`0b361220 OK inside mimidrv.sys
[03] IRP_MJ_READ fffff806`0b361220 OK inside mimidrv.sys
[04] IRP_MJ_WRITE fffff806`0b361220 OK inside mimidrv.sys
[05] IRP_MJ_QUERY_INFORMATION fffff806`0b361220 OK inside mimidrv.sys
[06] IRP_MJ_SET_INFORMATION fffff806`0b361220 OK inside mimidrv.sys
[07] IRP_MJ_QUERY_EA fffff806`0b361220 OK inside mimidrv.sys
[08] IRP_MJ_SET_EA fffff806`0b361220 OK inside mimidrv.sys
[09] IRP_MJ_FLUSH_BUFFERS fffff806`0b361220 OK inside mimidrv.sys
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION fffff806`0b361220 OK inside mimidrv.sys
[0b] IRP_MJ_SET_VOLUME_INFORMATION fffff806`0b361220 OK inside mimidrv.sys
[0c] IRP_MJ_DIRECTORY_CONTROL fffff806`0b361220 OK inside mimidrv.sys
[0d] IRP_MJ_FILE_SYSTEM_CONTROL fffff806`0b361220 OK inside mimidrv.sys
[0e] IRP_MJ_DEVICE_CONTROL fffff806`0b361434 OK inside mimidrv.sys
[0f] IRP_MJ_INTERNAL_DEVICE_CONTROL fffff806`0b361220 OK inside mimidrv.sys
[10] IRP_MJ_SHUTDOWN fffff806`0b361220 OK inside mimidrv.sys
[11] IRP_MJ_LOCK_CONTROL fffff806`0b361220 OK inside mimidrv.sys
[12] IRP_MJ_CLEANUP fffff806`0b361220 OK inside mimidrv.sys
[13] IRP_MJ_CREATE_MAILSLOT fffff806`0b361220 OK inside mimidrv.sys
[14] IRP_MJ_QUERY_SECURITY fffff806`0b361220 OK inside mimidrv.sys
[15] IRP_MJ_SET_SECURITY fffff806`0b361220 OK inside mimidrv.sys
[16] IRP_MJ_POWER fffff806`0b361220 OK inside mimidrv.sys
```

kd>

Locals Threads

El script ha confirmado automáticamente que:

Elemento	Resultado
<code>DRIVER_OBJECT</code>	Localizado correctamente en <code>fffffaa03f7443a80</code>
<code>DriverStart</code>	<code>fffff8060b360000</code>
<code>DriverSize</code>	<code>0xe000</code>
Rango del driver	<code>fffff8060b360000 - fffff8060b36e000</code>
<code>DriverUnload</code>	Dentro del rango de <code>mimidrv.sys</code>
<code>MajorFunction[00-1a]</code>	Dentro del rango de <code>mimidrv.sys</code>
<code>MajorFunction[0e]</code>	<code>IRP_MJ_DEVICE_CONTROL</code> , dentro del rango del driver
<code>MajorFunction[1b]</code>	<code>nt!IopInvalidDeviceRequest</code> , coherente para IRP no implementada

6.4. Conclusiones del funcionamiento del script

Para automatizar parte del análisis realizado manualmente con WinDbg, hemos desarrollado un script en JavaScript para el proveedor de scripting de WinDbg. El objetivo del script es localizar el `DRIVER_OBJECT` asociado a `\Driver\mimidrv`, obtener los campos `DriverStart` y `DriverSize`, calcular el rango de memoria ocupado por el driver y recorrer las 28 entradas de la tabla `MajorFunction`.

El script se ejecuta desde la máquina analista con los siguientes comandos:

```
.load jsprovider
.scriptload C:\Users\usuario2\Desktop\CheckMimidrvMajorFunction_clean.js
!checkmimidrv
```

La ejecución localiza correctamente el objeto de driver en la dirección `fffffaa03f7443a80`. A partir de la estructura `_DRIVER_OBJECT`, el script obtiene como dirección base del driver `fffff8060b360000` y como tamaño `0xe000`, calculando así el rango de memoria ocupado por `mimidrv.sys`:

```
fffff8060b360000 - fffff8060b36e000
```

Posteriormente, el script recorre la tabla `MajorFunction` ubicada en el offset `0x70` del `DRIVER_OBJECT`. El resultado muestra que la mayoría de entradas apuntan a la dirección `fffff8060b361220`, ubicada dentro del propio rango del driver. La entrada correspondiente a `IRP_MJ_DEVICE_CONTROL` apunta a `fffff8060b361434`, también dentro del rango de `mimidrv.sys`.

La última entrada, correspondiente a `IRP_MJ_PNP`, apunta a `nt!IopInvalidDeviceRequest`, una rutina estándar del kernel utilizada cuando un driver no implementa una determinada operación IRP. Este resultado se considera coherente.

La salida final del script es:

```
[+] Result: no suspicious MajorFunction pointers found.
```

Desde el punto de vista de detección, esto indica que no se han observado punteros sospechosos en la tabla `MajorFunction` del `DRIVER_OBJECT`. Es decir, no se han detectado rutinas que apunten fuera del rango esperado del driver ni a regiones de memoria desconocidas. Esta automatización permite repetir la comprobación de forma más rápida y sistemática, reduciendo la dependencia del análisis manual.

7. Conclusiones

La práctica ha permitido comprobar cómo WinDbg puede utilizarse para analizar componentes cargados en Ring 0 desde una máquina analista. Mediante la depuración remota se ha podido observar el driver `mimidrv.sys`, identificar su objeto asociado en el kernel y revisar su estructura `DRIVER_OBJECT`.

El análisis manual permitió comprobar la tabla `MajorFunction`, las rutinas principales del driver y el rango de memoria en el que se encontraba cargado. Posteriormente, mediante un script en JavaScript para WinDbg, se automatizó la revisión de estas rutinas para detectar posibles punteros fuera del rango esperado del driver.

7.1 Qué se puede detectar

Con este enfoque es posible detectar drivers cargados en el kernel, objetos presentes en el namespace `\Driver`, rutinas de despacho asociadas a un driver y posibles anomalías en la tabla `MajorFunction`. En concreto, se pueden identificar punteros que apunten fuera del rango de memoria del propio driver, hacia módulos inesperados o hacia regiones desconocidas, lo que podría indicar hooking o manipulación del `DRIVER_OBJECT`.

También permite localizar rutinas especialmente relevantes, como `IRP_MJ_DEVICE_CONTROL`, que suele gestionar la comunicación mediante IOCTL entre procesos de usuario y drivers de kernel.

7.2 Limitaciones

La revisión de `MajorFunction` permite detectar incoherencias estructurales, pero no determina por sí sola si un driver es malicioso. Un driver puede tener sus rutinas dentro del rango esperado y aun así realizar acciones peligrosas desde Ring 0.

Además, este análisis depende de que el entorno de depuración esté correctamente configurado, de que el driver esté cargado en el momento de la inspección y de que los símbolos funcionen correctamente. También existen técnicas de ocultación o manipulación en kernel que no se detectan únicamente revisando el `DRIVER_OBJECT`.

En conclusión, WinDbg es una herramienta muy útil para analizar actividad en Ring 0 y detectar anomalías en drivers, pero sus resultados deben interpretarse junto con otras evidencias del sistema, como eventos, servicios, módulos cargados, hashes y comportamiento observado.